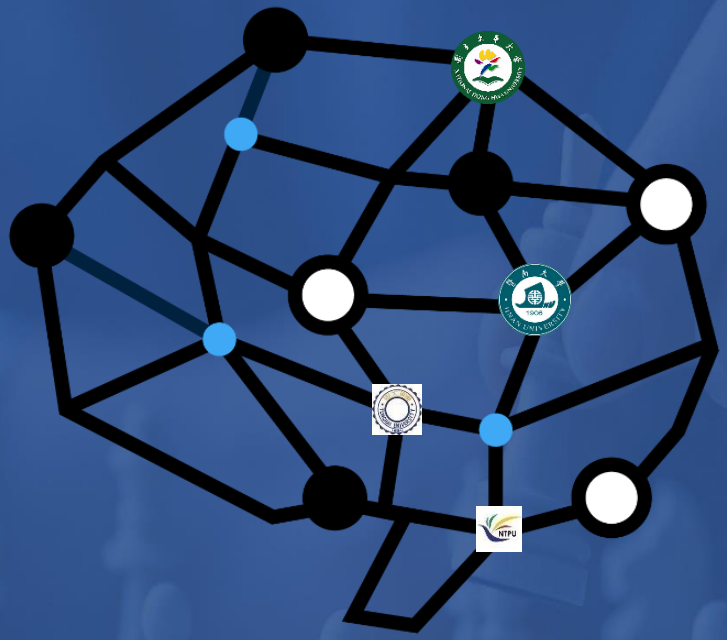
教育部課程改進計畫
人工智慧技術與應用 - 最先進之電腦對局遊戲系統設計



Chap. 5 Game Playing

課程：人工智慧導論

授課教師：周信宏

DeepBlue

Deep Blue

Kasparov

Outline

- Games vs. search problems
- Minimax tree
- Alpha-Beta search
- Game of chance
- Games of imperfect information
- Solving games

Games vs. search problems

"Unpredictable" opponent $\Rightarrow$ solution is a strategy specifying a move for every possible opponent reply

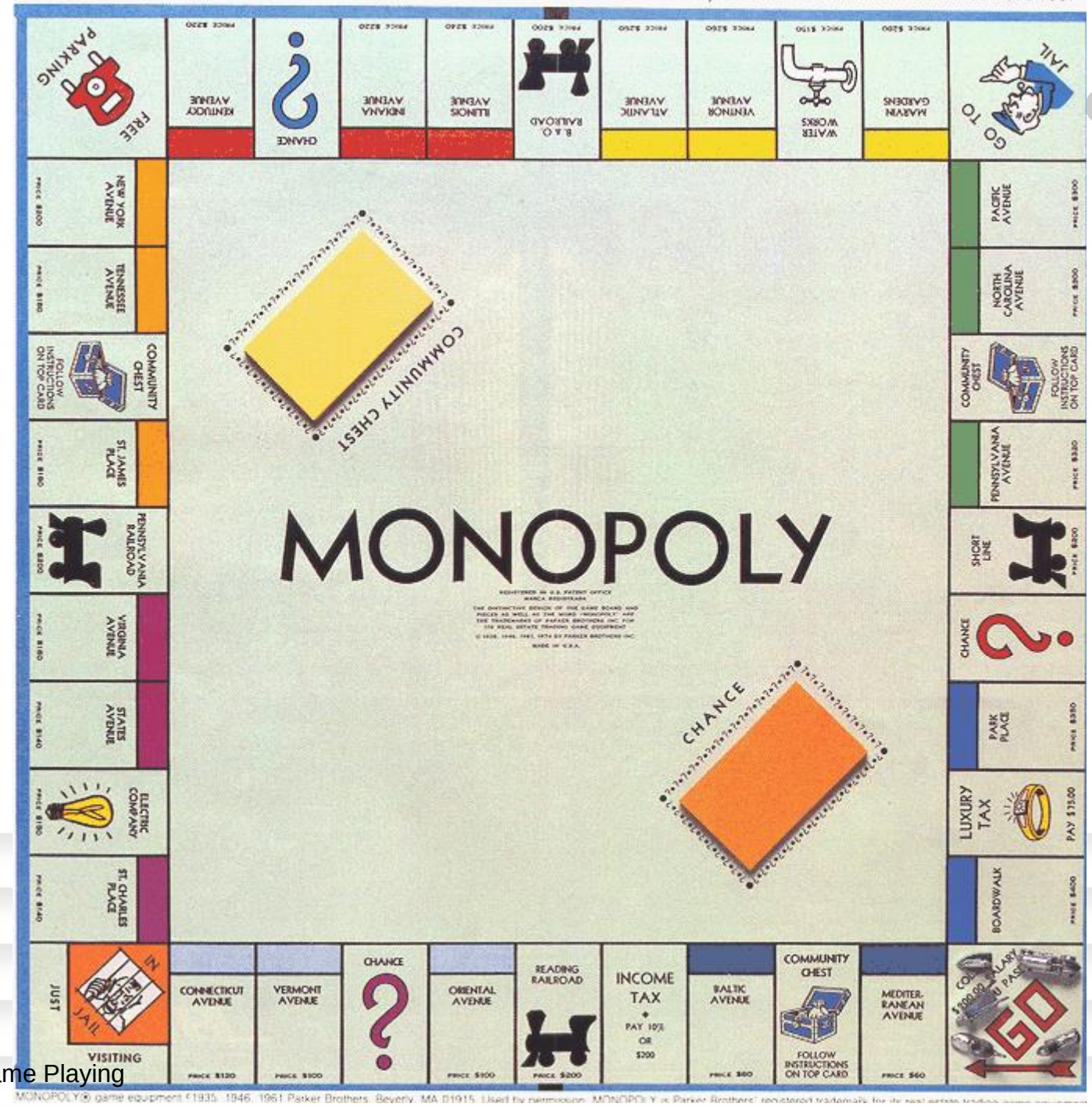
Time limits $\Rightarrow$ unlikely to find goal, must approximate

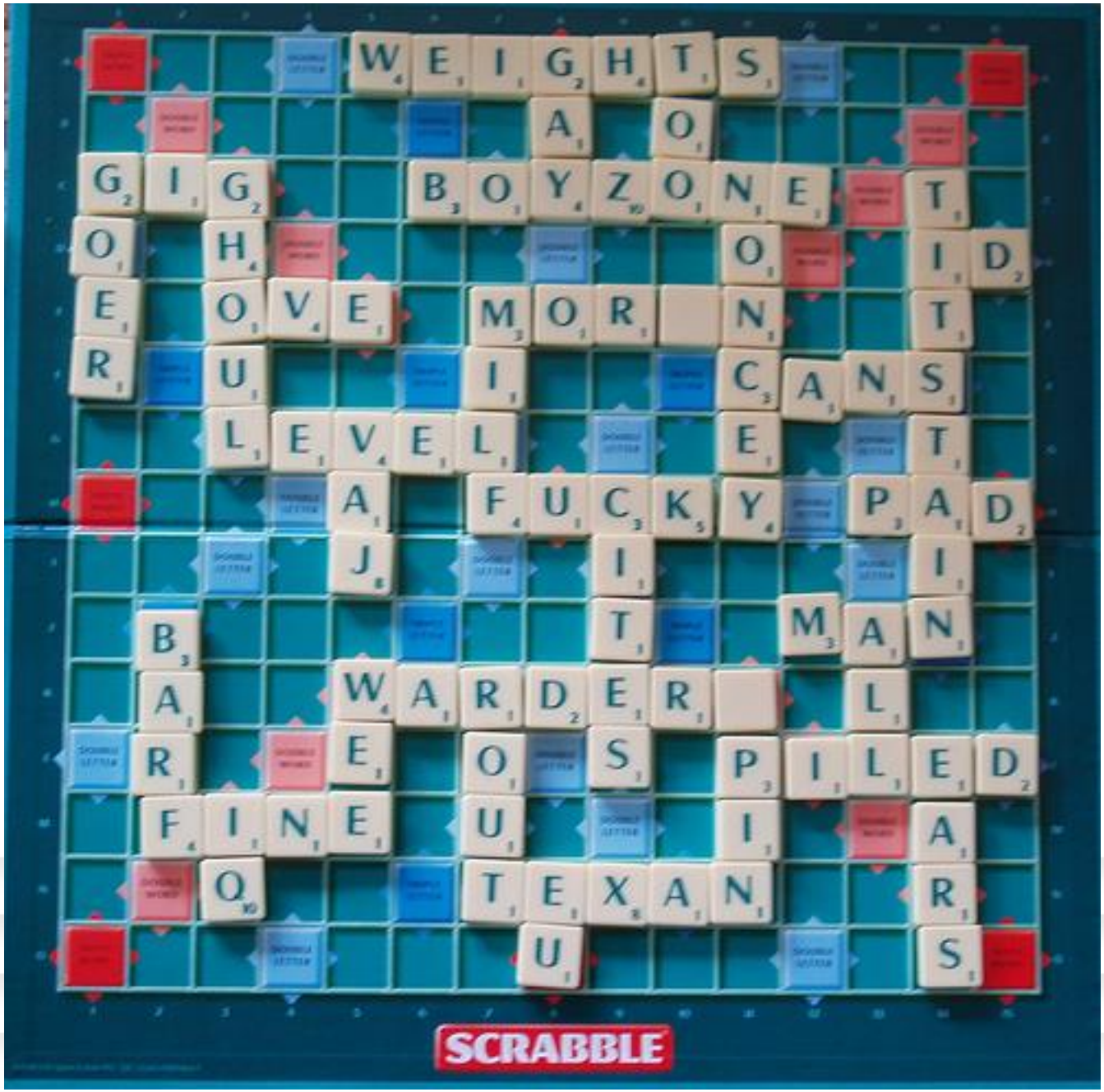
Plan of attack:

- Computer considers possible lines of play (Babbage, 1846)
- Algorithm for perfect play (Zermelo, 1912; Von Neumann, 1944)
- Finite horizon, approximate evaluation (Zuse, 1945; Wiener, 1948; Shannon, 1950)
- First chess program (Turing, 1951)
- Machine learning to improve evaluation accuracy (Samuel, 1952–57)
- Pruning to allow deeper search (McCarthy, 1956)

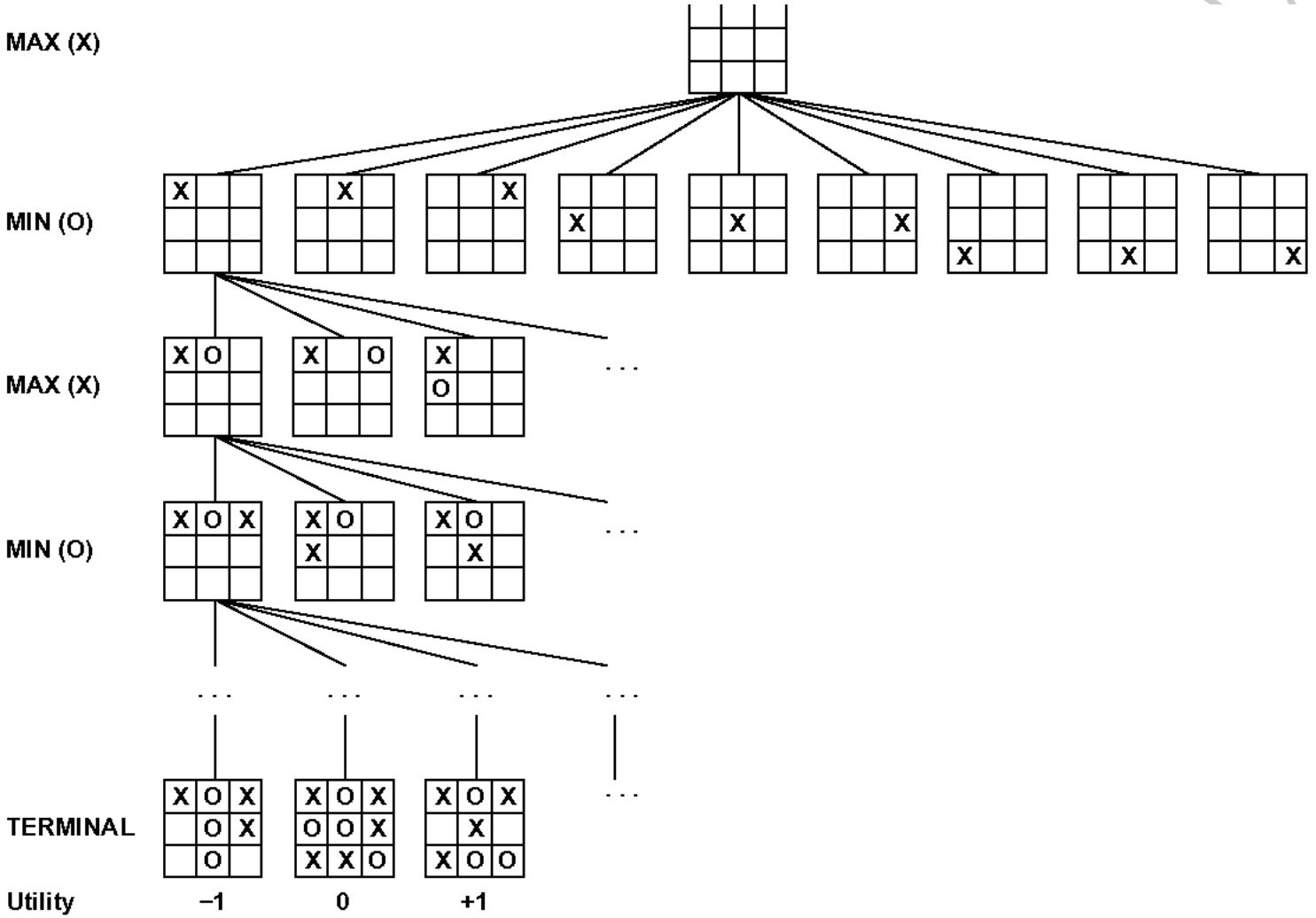
Types of games

	deterministic	chance
perfect information	chess, checkers, go, othello	backgammon monopoly
imperfect information		bridge, poker, scrabble nuclear war


MONOPOLY
COMMUNITY CHEST
CHANCE
GO
JAIL
FREE PARKING
GO TO JAIL
MARVIN GARDENS
WATER WORKS
VENTNOR AVENUE
ATLANTIC AVENUE
B. & O. RAILROAD
ILLINOIS AVENUE
INDIANA AVENUE
CHANCE
KENTUCKY AVENUE
NEW YORK AVENUE
TENNESSEE AVENUE
COMMUNITY CHEST
ST. JAMES PLACE
PENNSYLVANIA RAILROAD
VIRGINIA AVENUE
STATES AVENUE
ELECTRIC COMPANY
ST. CHARLES PLACE
JUST VISITING IN JAIL
CONNECTICUT AVENUE
VERMONT AVENUE
CHANCE
ORIENTAL AVENUE
READING RAILROAD
INCOME TAX
BALTIC AVENUE
COMMUNITY CHEST
MEDITERRANEAN AVENUE
PACIFIC AVENUE
NORTH CAROLINA AVENUE
COMMUNITY CHEST
PENNSYLVANIA AVENUE
SHORT LINE
CHANCE
PARK PLACE
LUXURY TAX
BOARDWALK


WEIGHTS
AO
GIG
BOYZONE
OHOT
OVEMORID
ERUICANT
LEVELEST
AFUCKYPAD
JIT
BMAN
AWARDER
RESPILED
FINEUI
QTEXANAR
US
SCRABBLE

Game tree of Tic-Tac-Toe (2-player, deterministic, turns)

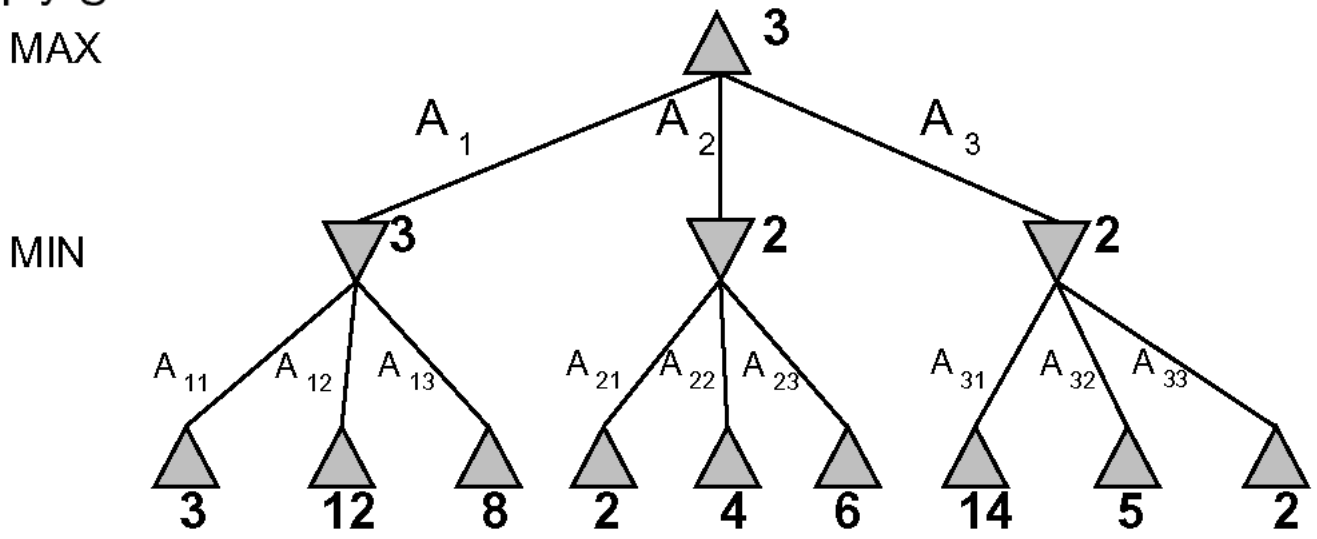


Minimax

Perfect play for deterministic, perfect-information games

Idea: choose move to position with highest *minimax value*
= best achievable payoff against best play

E.g., 2-ply game:



Minimax algorithm

function MINIMAX-DECISION($state, game$) **returns** *an action*

$action, state \leftarrow$ the a, s **in** SUCCESSORS(s_{state})
such that MINIMAX-VALUE($s, game$) is maximized

return *action*

---

function MINIMAX-VALUE($s_{state}, game$) **returns** *a utility value*

if TERMINAL-TEST($state$) **then**
return UTILITY(s_{state})
else if MAX is to move in *state* **then**
return the highest MINIMAX-VALUE of SUCCESSORS($state$)
else
return the lowest MINIMAX-VALUE of SUCCESSORS(s_{state})

Analysis

Complete??

Yes, if tree is finite (chess has specific rules for this)

Optimal?? Yes, against an optimal opponent. Otherwise??

Time complexity?? $O(b^m)$

Space complexity?? $O(bm)$ (depth-first exploration)

For chess, $b \approx 35$, $m \approx 100$ for "reasonable" games

$\Rightarrow$ exact solution completely infeasible

Resource limits

Suppose we have 100 seconds, explore 10^4 nodes/second

$\Rightarrow 10^6$ nodes per move

Standard approach:

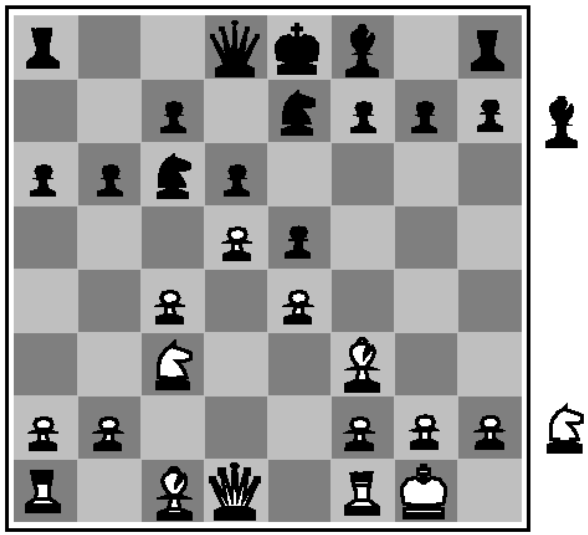
- *cutoff test*

  e.g., depth limit (perhaps add *quiescence search*)

- *evaluation function*

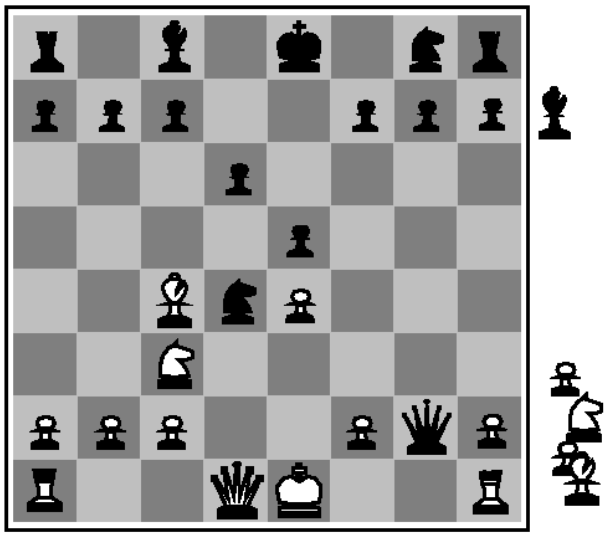
  = estimated desirability of position

Evaluation functions



Black to move

White slightly better



White to move

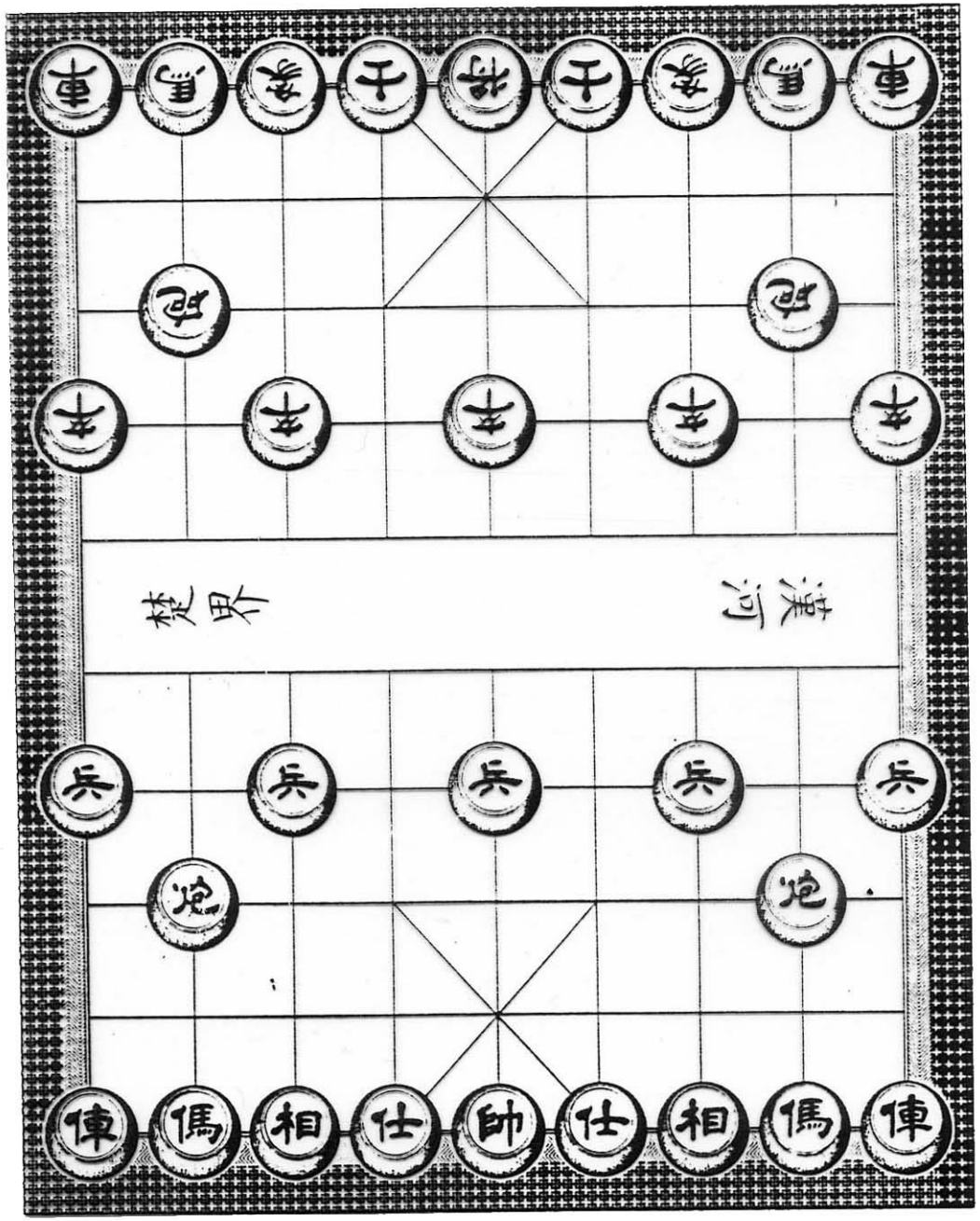
Black winning

For chess, typically *linear* weighted sum of features

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \ldots + w_n f_n(s)$$

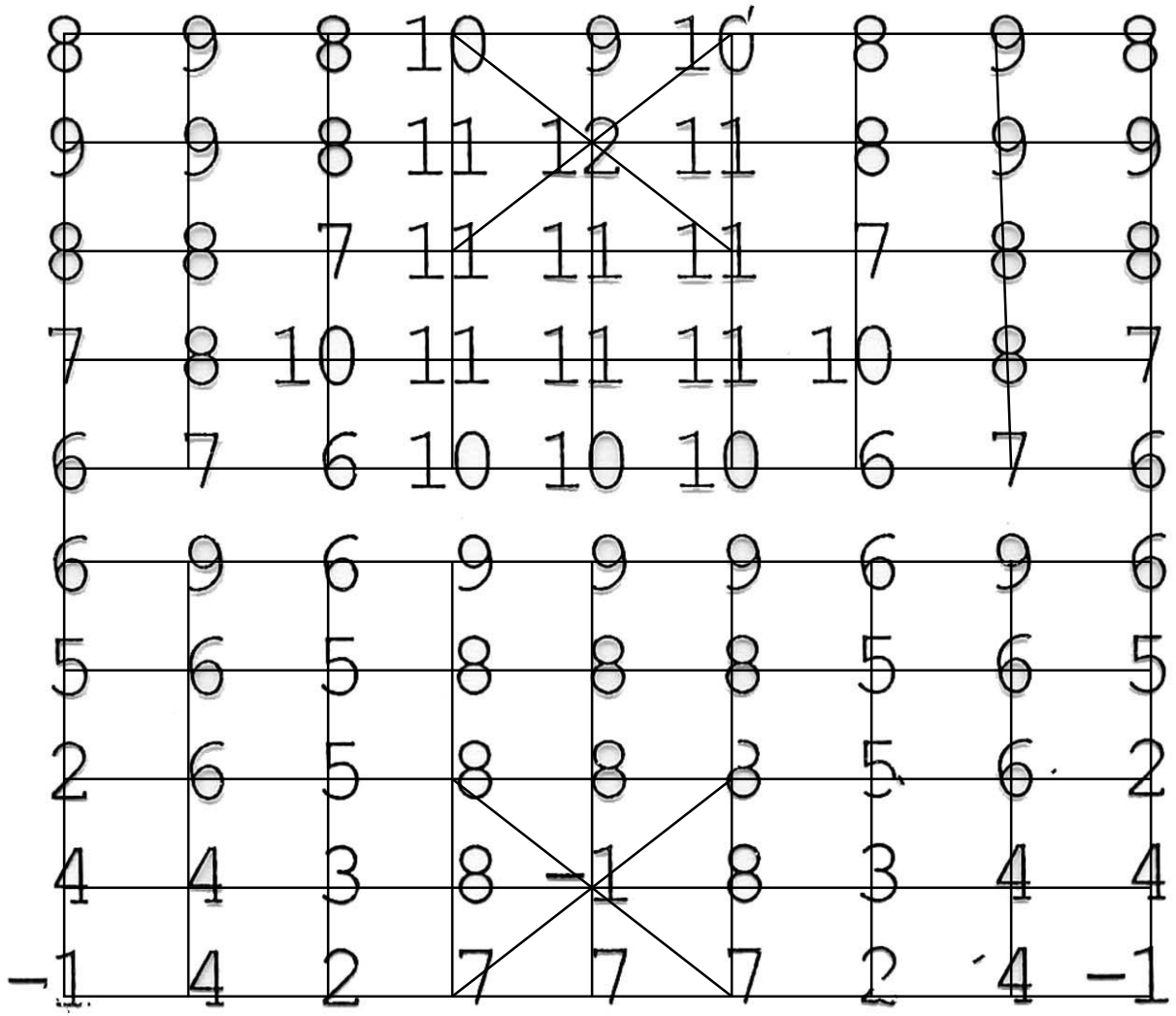
e.g., $w_1 = 9$ with
$f_1(s) =$ (number of white queens) – (number of black queens), etc.

1. 子力
2. 位置



子力

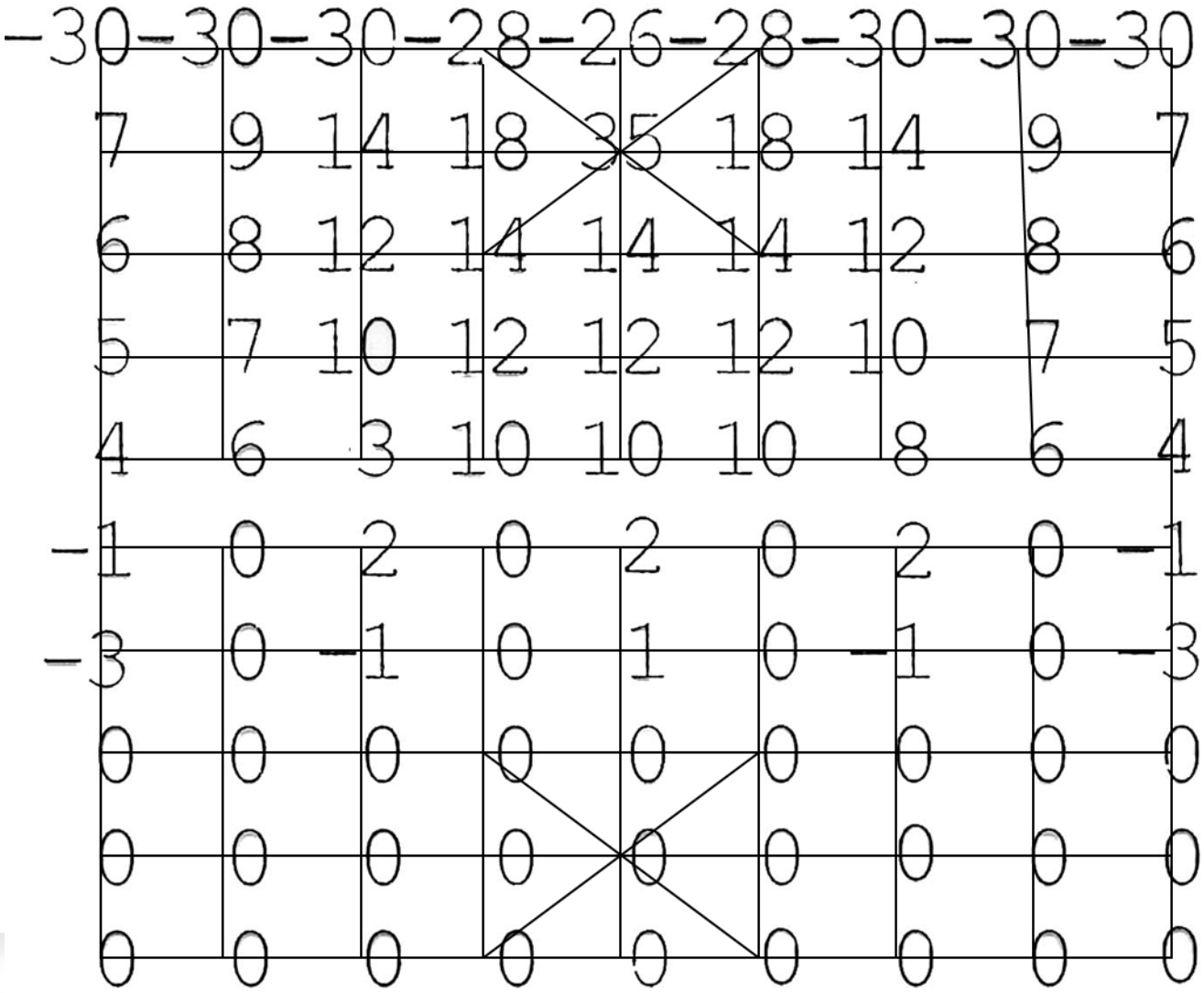
將	2000
士	40
象	40
車	200
馬	90
包	90
卒	10


8 9 8 10 9 10 8 9 8
9 9 8 11 12 11 8 9 9
8 8 7 11 11 11 7 8 8
7 8 10 11 11 11 10 8 7
6 7 6 10 10 10 6 7 6
6 9 6 9 9 9 6 9 6
5 6 5 8 8 8 5 6 5
2 6 5 8 8 8 5 6 2
4 4 3 8 -1 8 3 4 4
-1 4 2 7 7 7 2 4 -1

車

4	3	3	6	3	6	3	3	4
3	8	8	6	3	6	8	8	3
7	6	8	8	8	8	8	6	7
4	9	6	9	6	9	6	9	4
4	7	6	7	8	7	6	7	4
0	3	6	4	4	4	6	3	0
0	3	3	2	4	2	3	3	0
0	2	2	2	0	2	2	2	0
0	0	0	1	−1	0	1	0	0	0
0	−2	0	0	0	0	0	−2	0

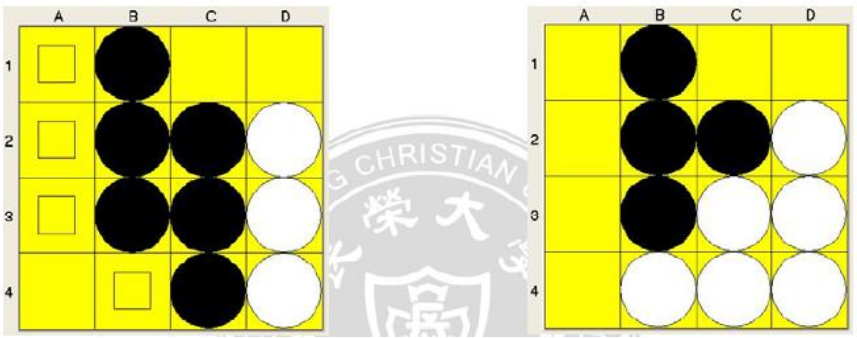
馬

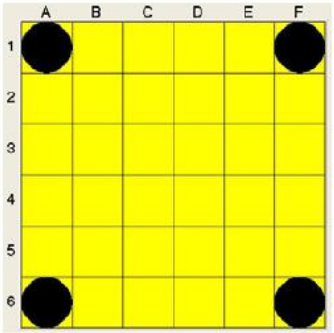

−30 −30 −30 −28 −26 −28 −30 −30 −30
7 9 14 18 35 18 14 9 7
6 8 12 14 14 14 12 8 6
5 7 10 12 12 12 10 7 5
4 6 3 10 10 10 8 6 4
−1 0 2 0 2 0 2 0 −1
−3 0 −1 0 1 0 −1 0 −3
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0

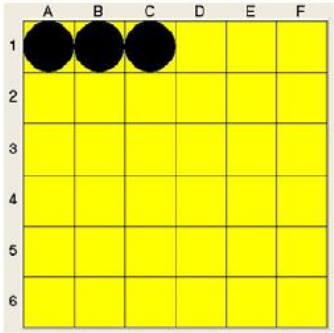
卒

Othello

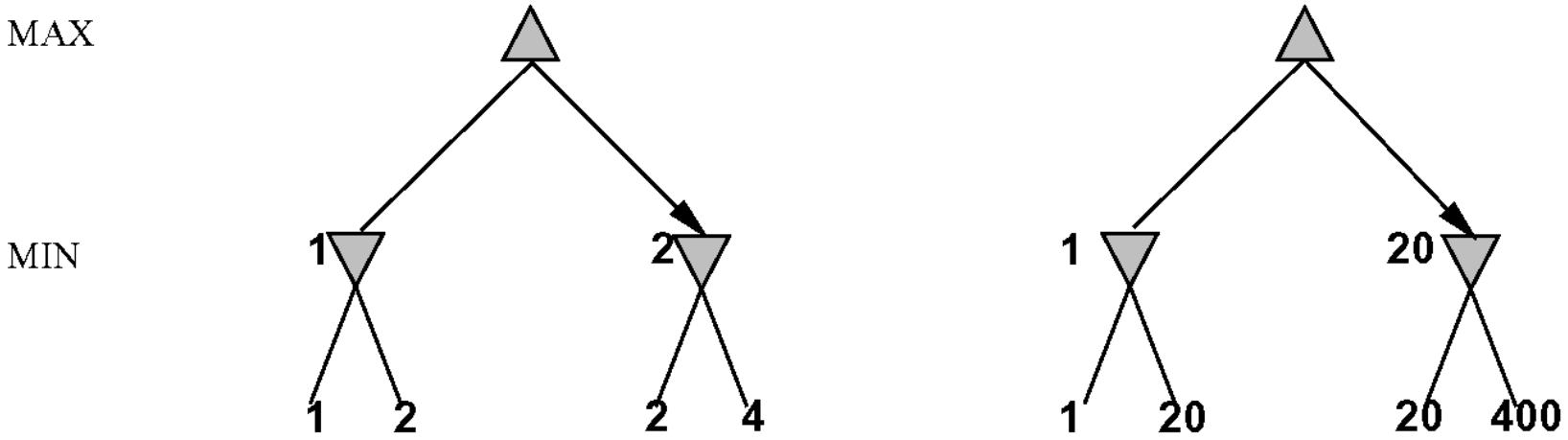
- 最多子(Maximum Pieces)
- 位置權重(Weighted Square)
- 行動能力(Mobility Consideration)
- 穩定子(Stability Pieces)







Digression: Exact values don't matter



Behaviour is preserved under any *monotonic* transformation of EVAL

Only the order matters:
payoff in deterministic games acts as an *ordinal utility* function

Cutting off search

MinimaxCutoff is identical to MinimaxValue except

1. Terminal? is replaced by Cutoff?
2. Utility is replaced by Eval

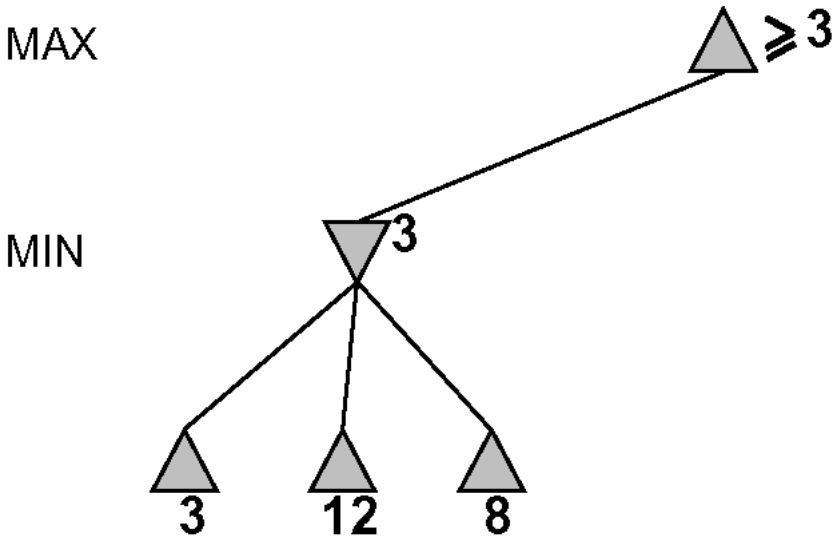
Does it work in practice?

$$b^m = 10^6, \quad b = 35 \quad \Rightarrow \quad m = 4$$

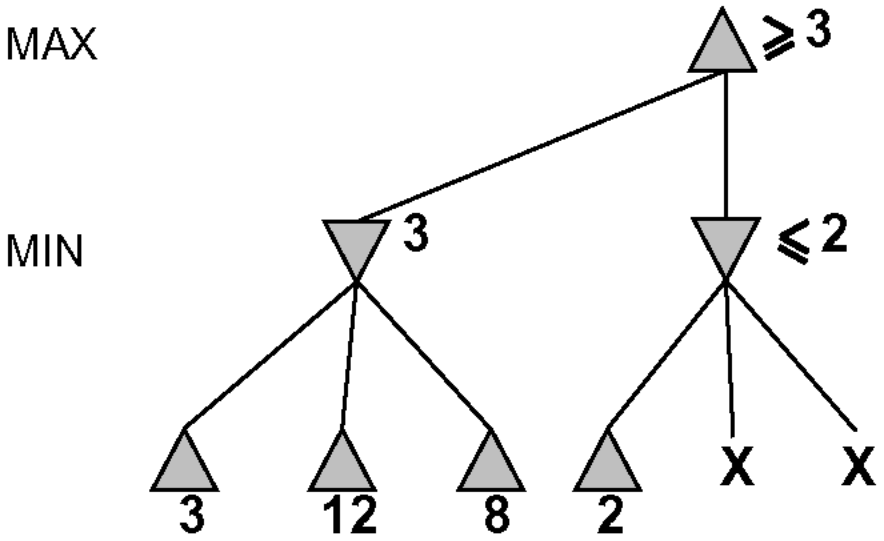
4-ply lookahead is a hopeless chess player!

4-ply $\approx$ human novice
8-ply $\approx$ typical PC, human master
12-ply $\approx$ Deep Blue, Kasparov

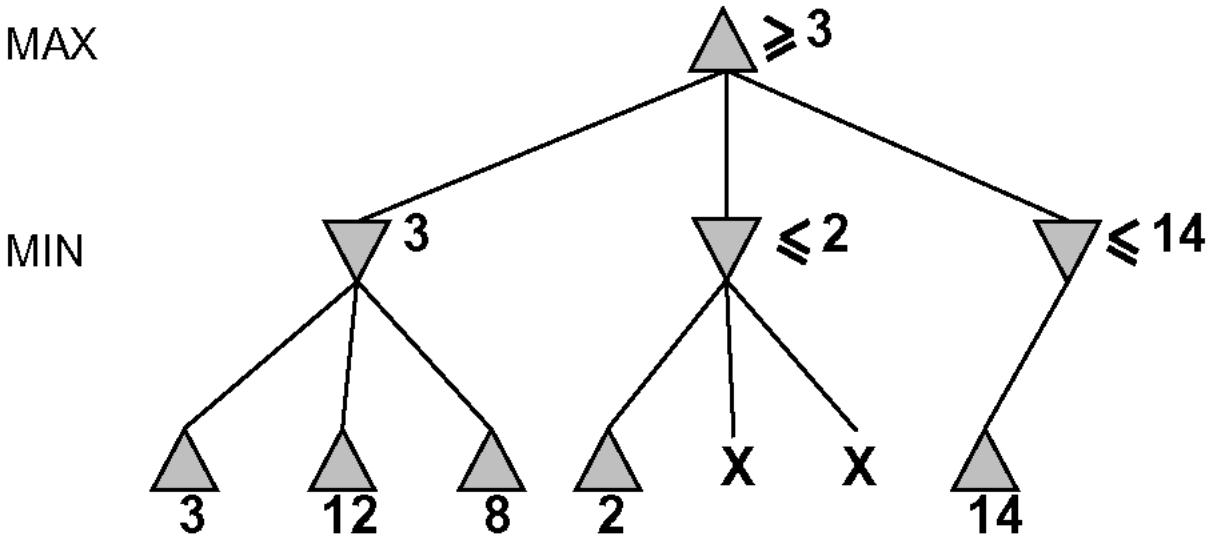
α–β pruning example



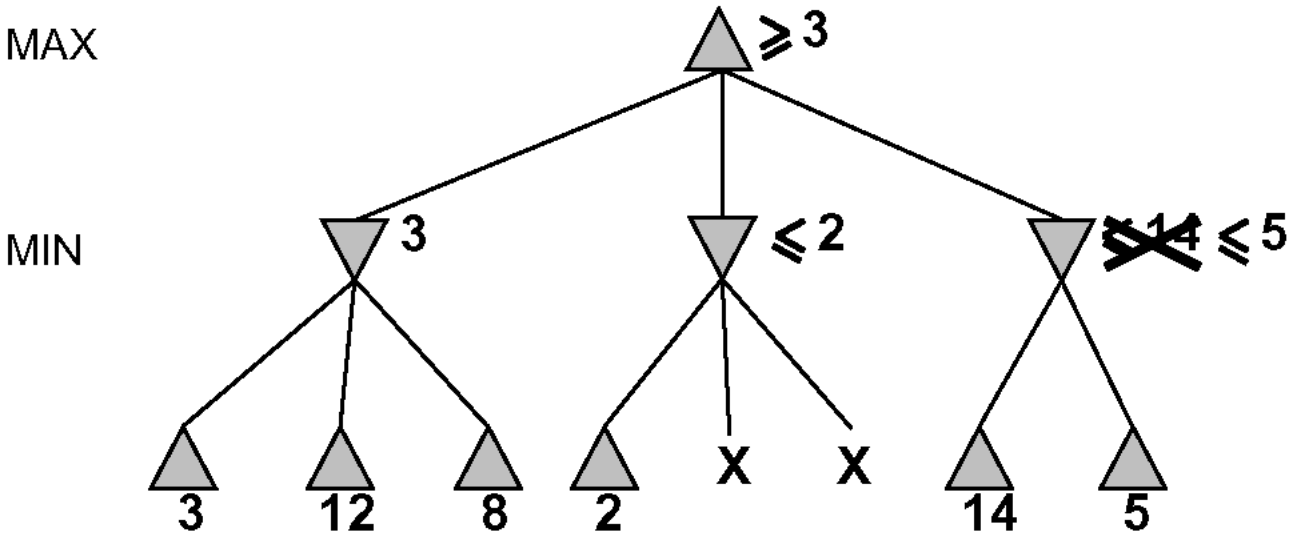
α–β pruning example



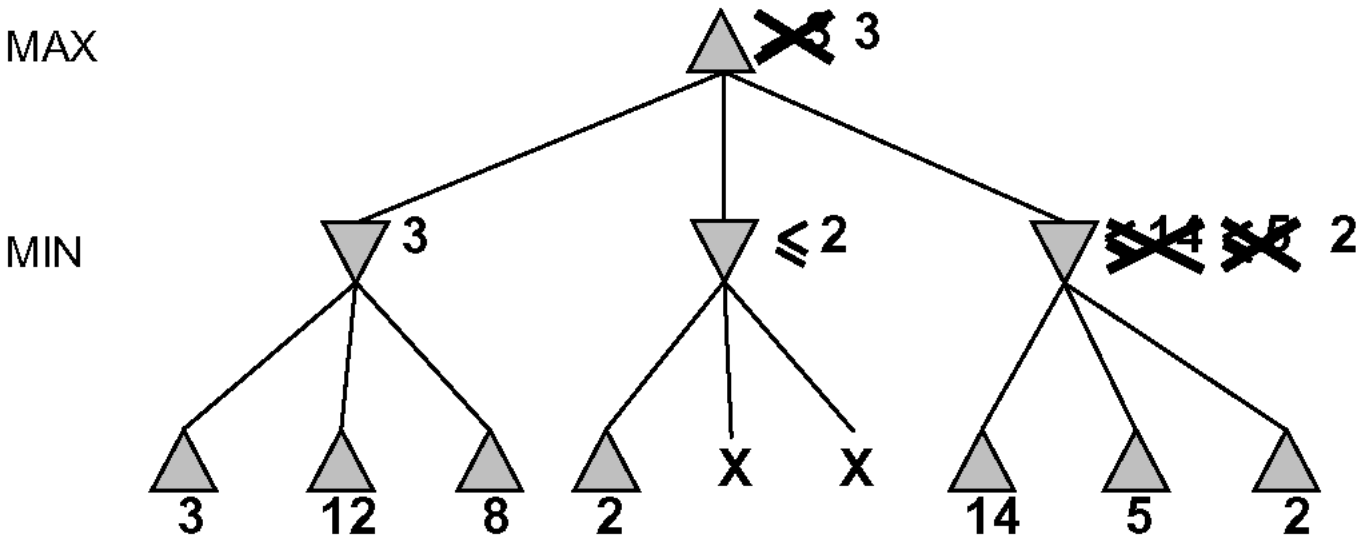
α–β pruning example



α–β pruning example



α–β pruning example



Properties of α–β

Pruning *does not* affect final result

Good move ordering improves effectiveness of pruning

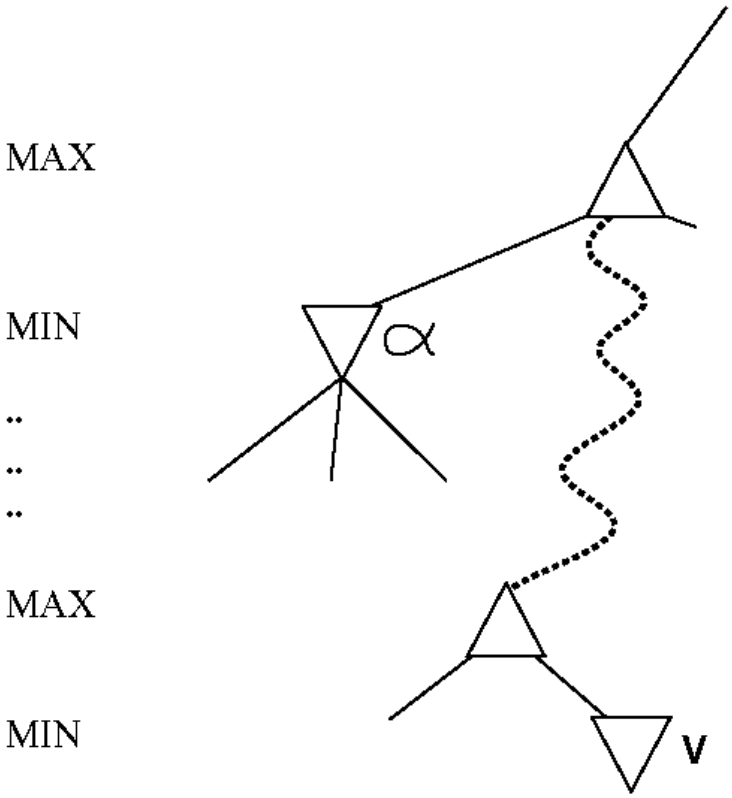
With "perfect ordering," time complexity $= O(b^{m/2})$

$\Rightarrow$ *doubles* depth of search

$\Rightarrow$ can easily reach depth 8 and play good chess

A simple example of the value of reasoning about which computations are relevant (a form of *metareasoning*)

Why is it called α–β?



α is the best value (to MAX) found so far off the current path

If V is worse than α, MAX will avoid it $\Rightarrow$ prune that branch

Define β similarly for MIN

The α–β algorithm

function ALPHA-BETA-SEARCH($state, game$) **returns** an action
  $action, state \leftarrow$ the a, s **in** SUCCESSORS[$game$](s_{state})
    such that MIN-VALUE($s, game, -\infty, +\infty$) is maximized
  return $action$

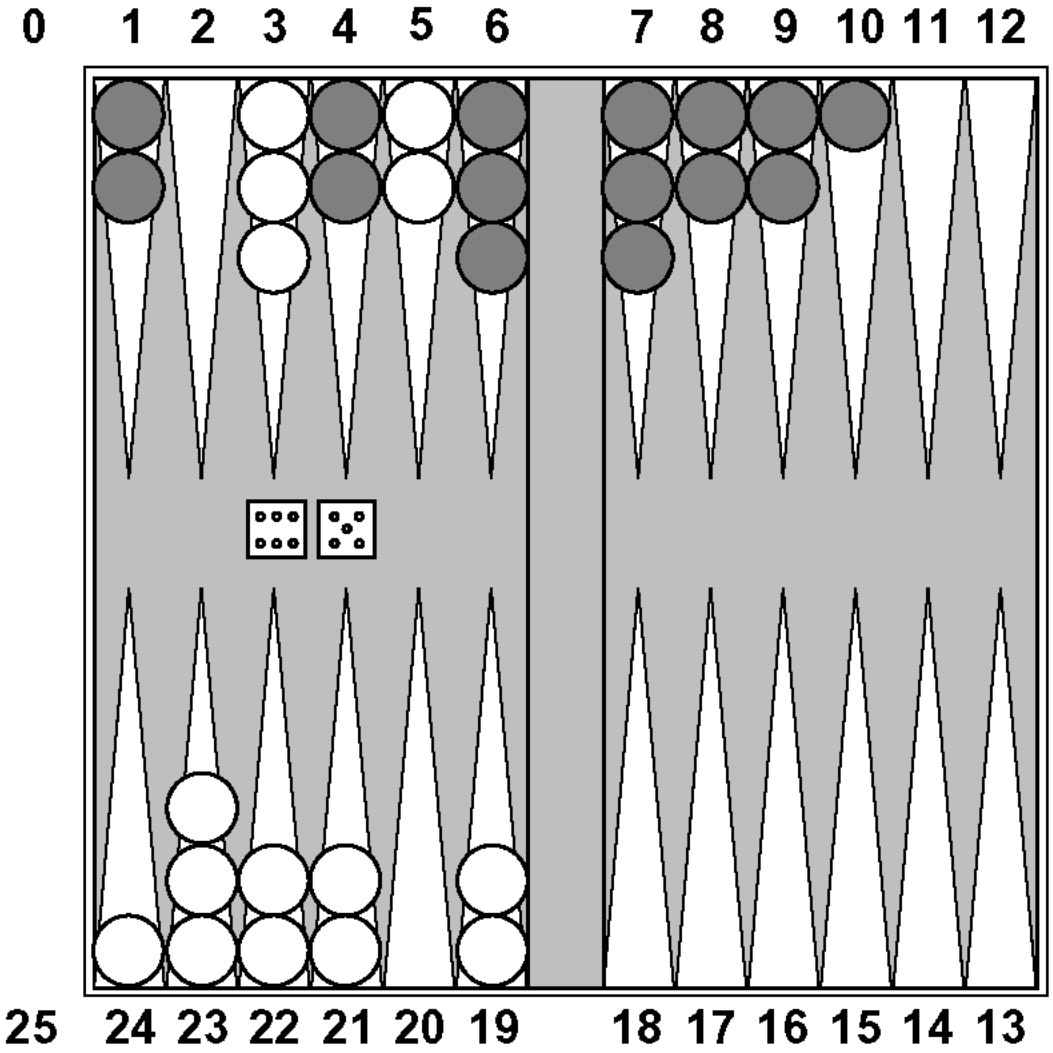
---

function MAX-VALUE($state, game, \alpha, \beta$) **returns** the minimax value of $state$
  if CUTOFF-TEST(s_{state}) **then return** EVAL($state$)
  for each s **in** SUCCESSORS($state$) **do**
    $\alpha \leftarrow \max(\alpha,$ MIN-VALUE($s, game, \alpha, \beta$))
    if $\alpha \geq \beta$ **then return** β
  return α

---

function MIN-VALUE($state, game, \alpha, \beta$) **returns** the minimax value of $state$
  if CUTOFF-TEST(s_{state}) **then return** EVAL($state$)
  for each s **in** SUCCESSORS($state$) **do**
    $\beta \leftarrow \min(\beta,$ MAX-VALUE($s, game, \alpha, \beta$))
    if $\beta \leq \alpha$ **then return** α
  return β

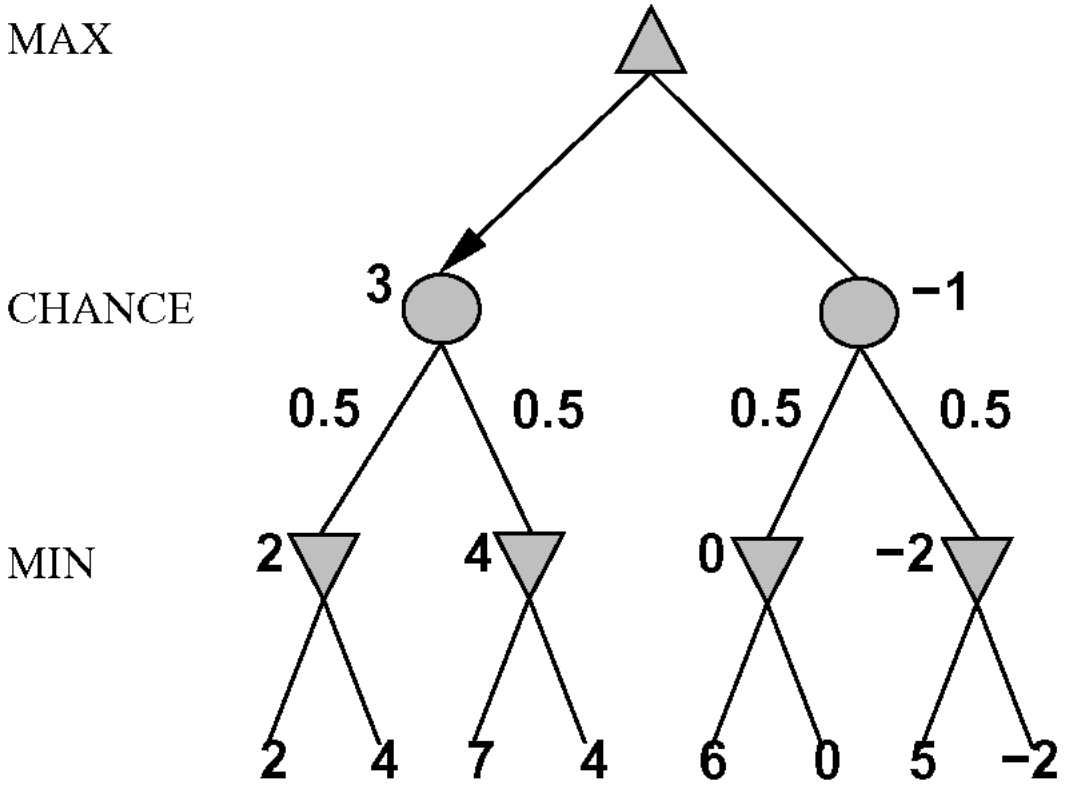
Nondeterministic games: backgammon



Nondeterministic games in general

In nondeterministic games, chance introduced by dice, card-shuffling

Simplified example with coin-flipping:



Algorithm for nondeterministic games

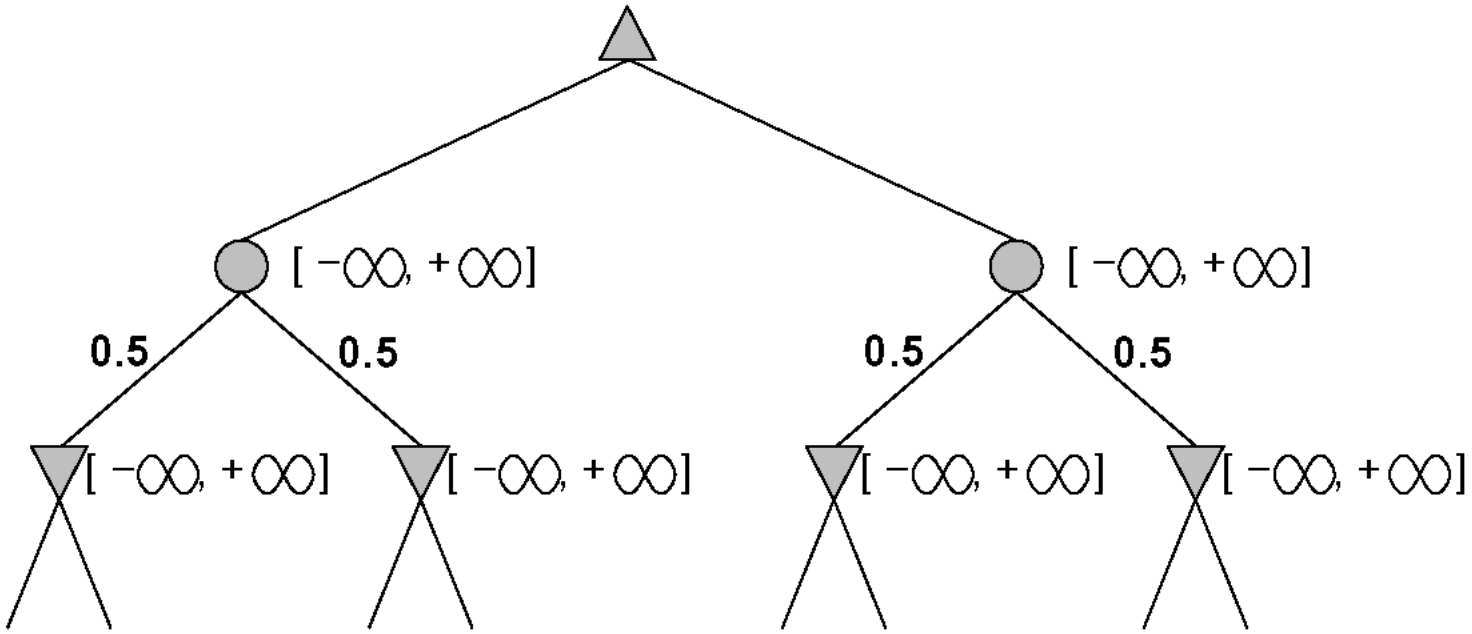
EXPECTIMINIMAX gives perfect play

Just like MINIMAX, except we must also handle chance nodes:

```
...
if state is a MAX node then
        return the highest EXPECTIMINIMAX-VALUE of SUCCESSORS(state)
if state is a MIN node then
        return the lowest EXPECTIMINIMAX-VALUE of SUCCESSORS(state)
if state is a chance node then
        return average of EXPECTIMINIMAX-VALUE of SUCCESSORS(state)
...
```

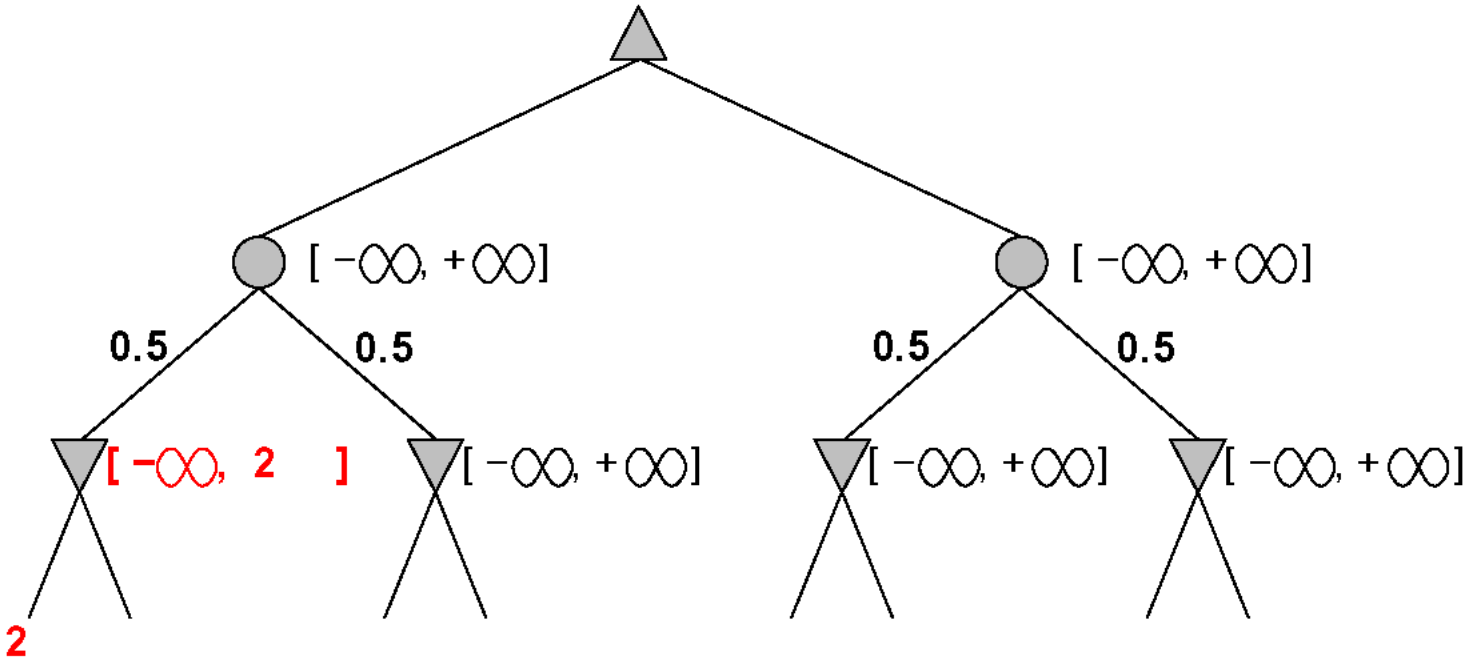
Pruning in nondeterministic game trees

A version of α-β pruning is possible:



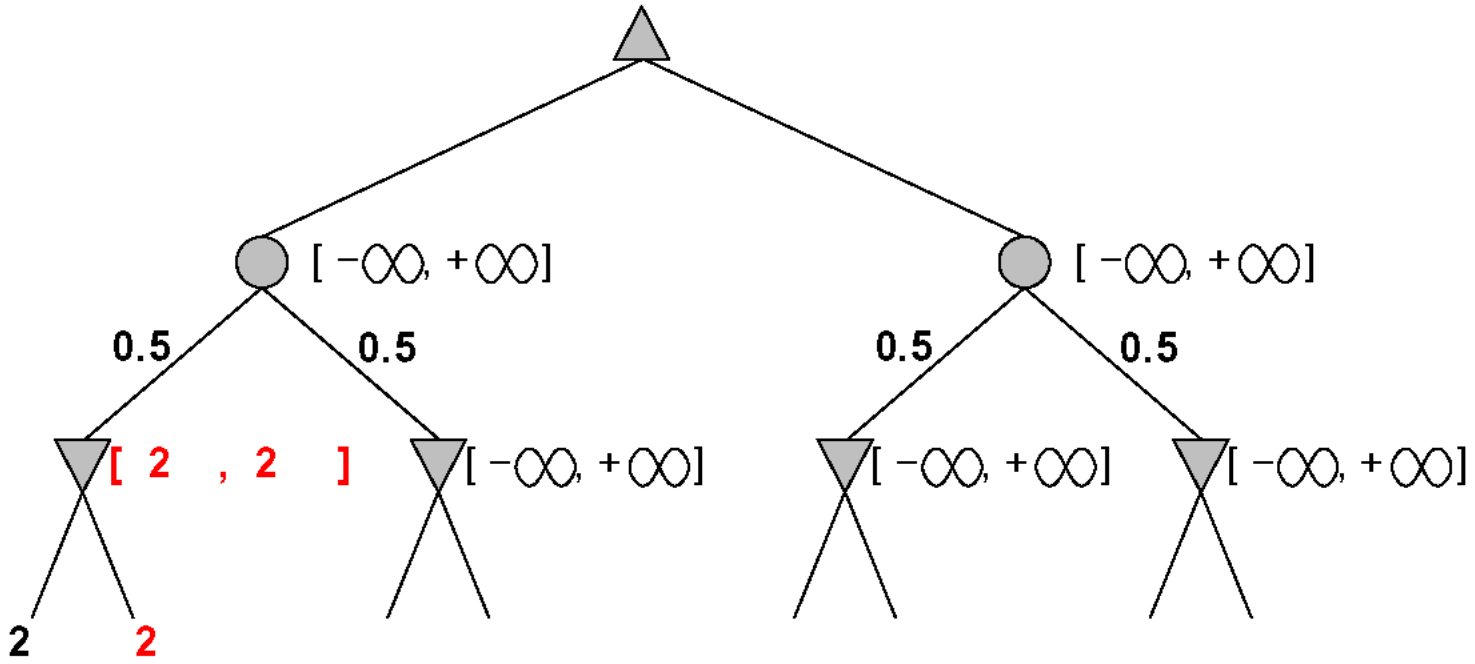
Pruning in nondeterministic game trees

A version of α-β pruning is possible:



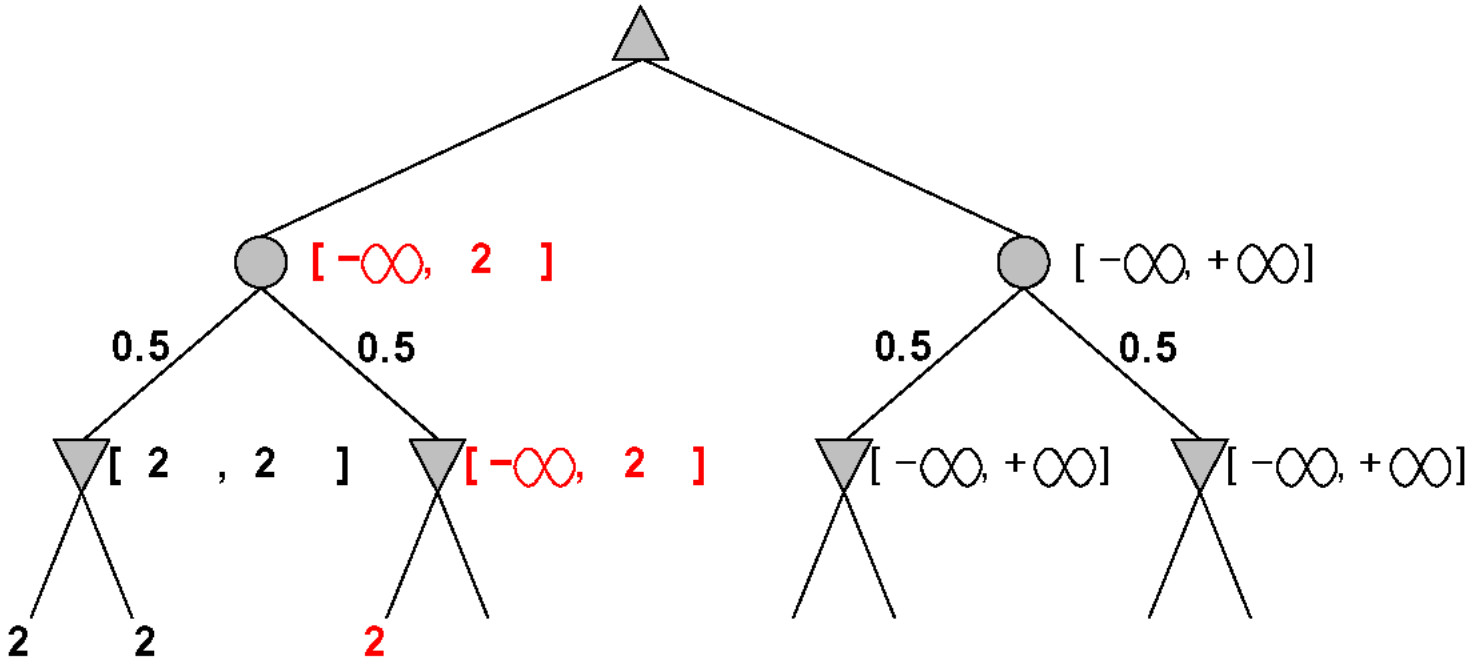
Pruning in nondeterministic game trees

A version of α-β pruning is possible:



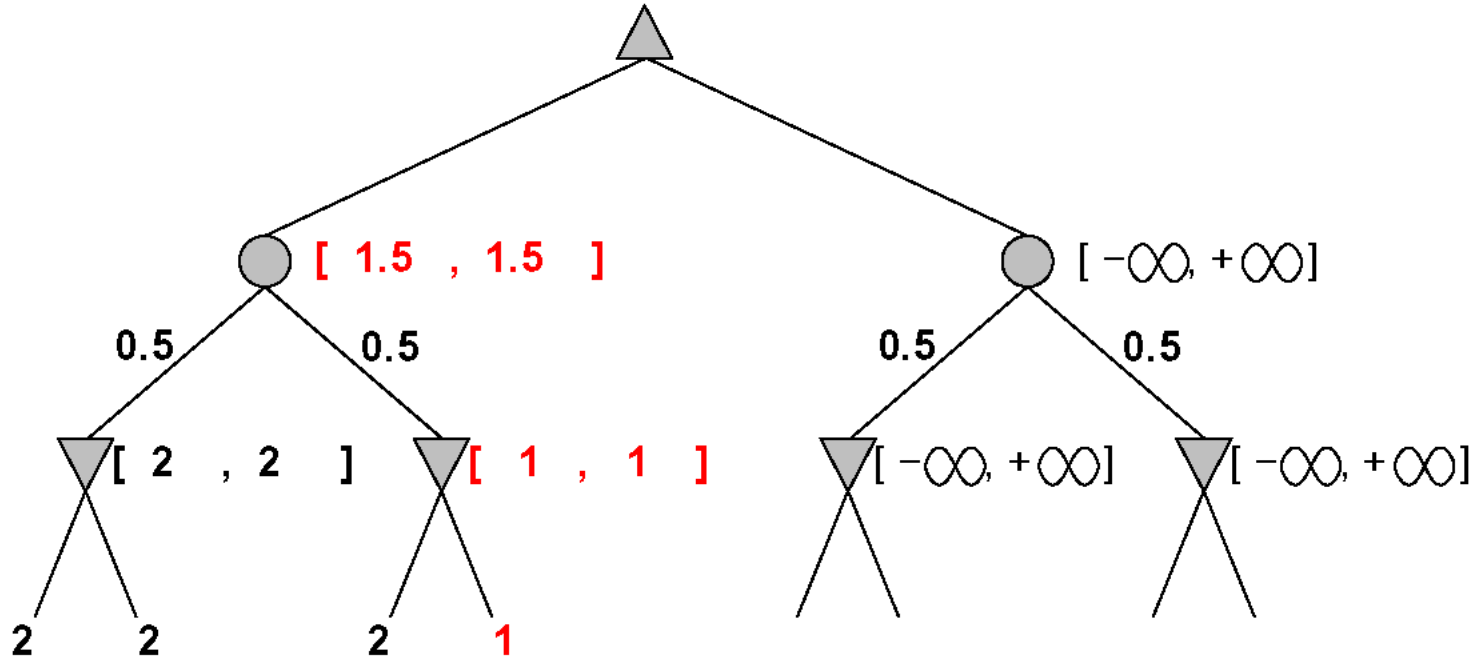
Pruning in nondeterministic game trees

A version of α-β pruning is possible:



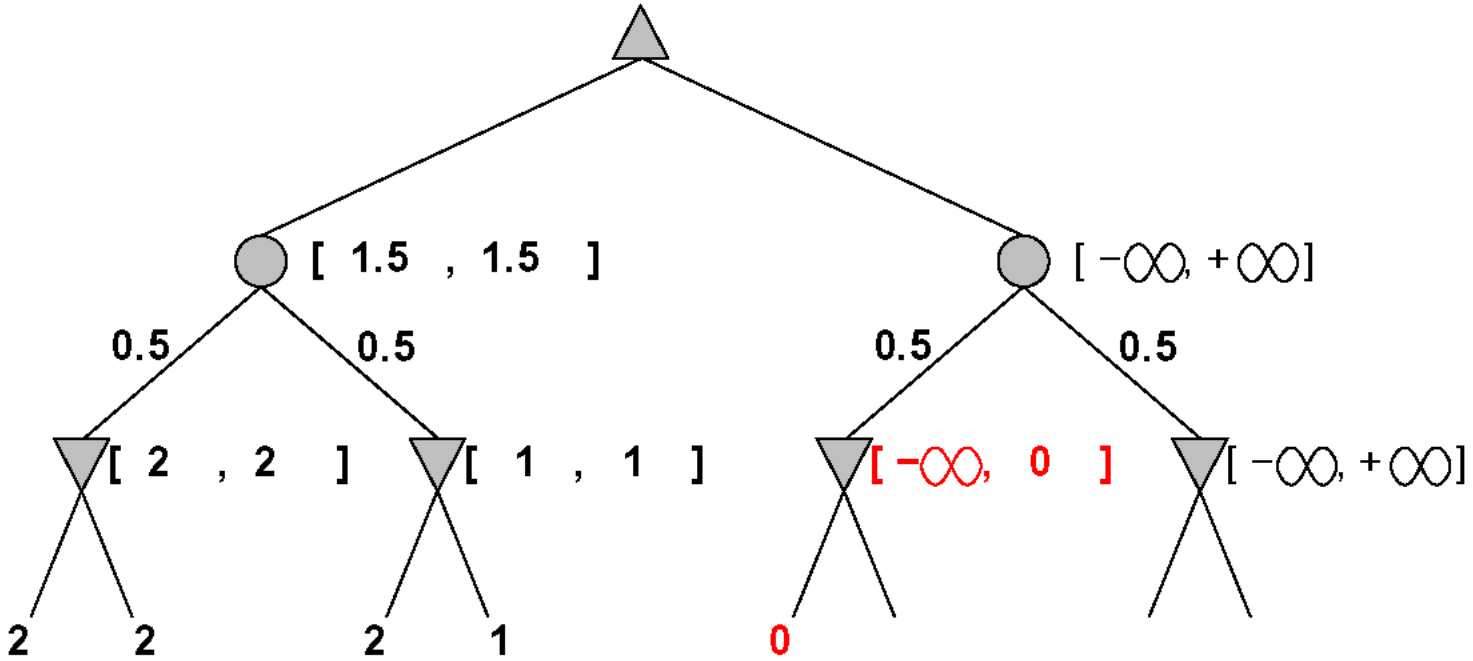
Pruning in nondeterministic game trees

A version of α-β pruning is possible:



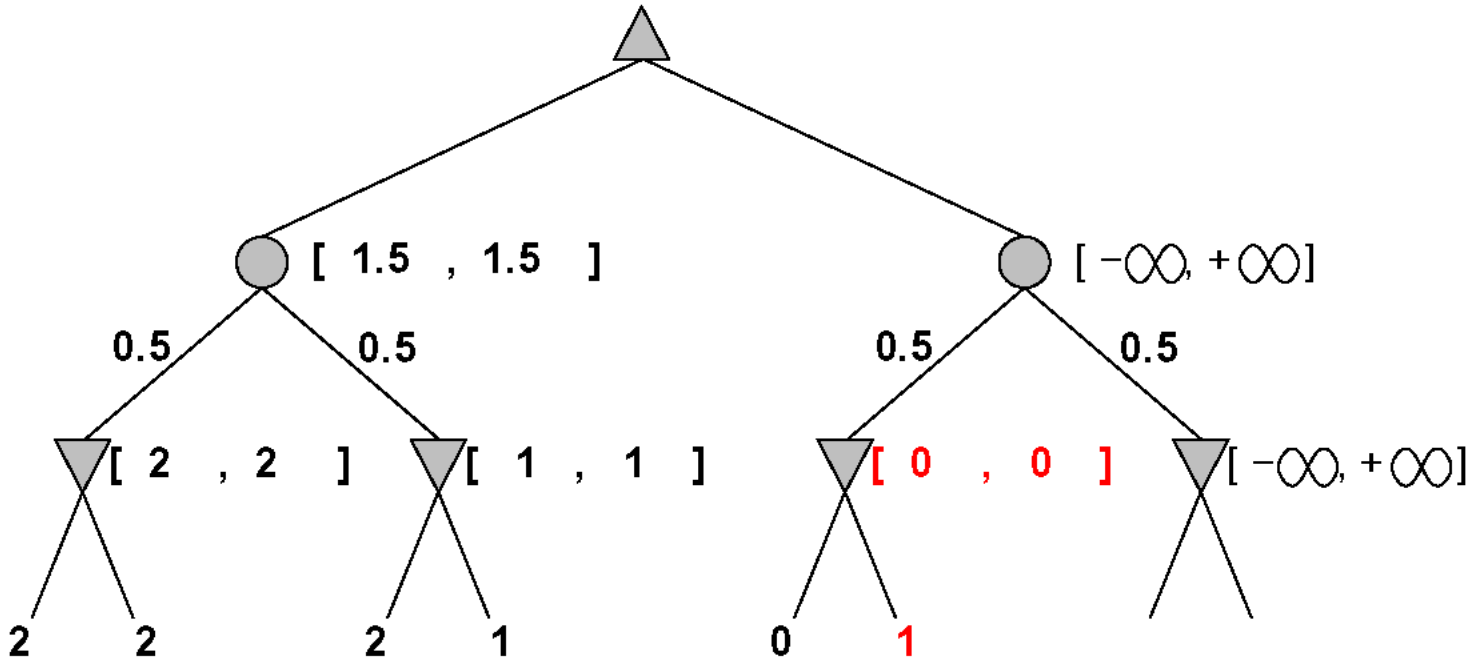
Pruning in nondeterministic game trees

A version of α-β pruning is possible:



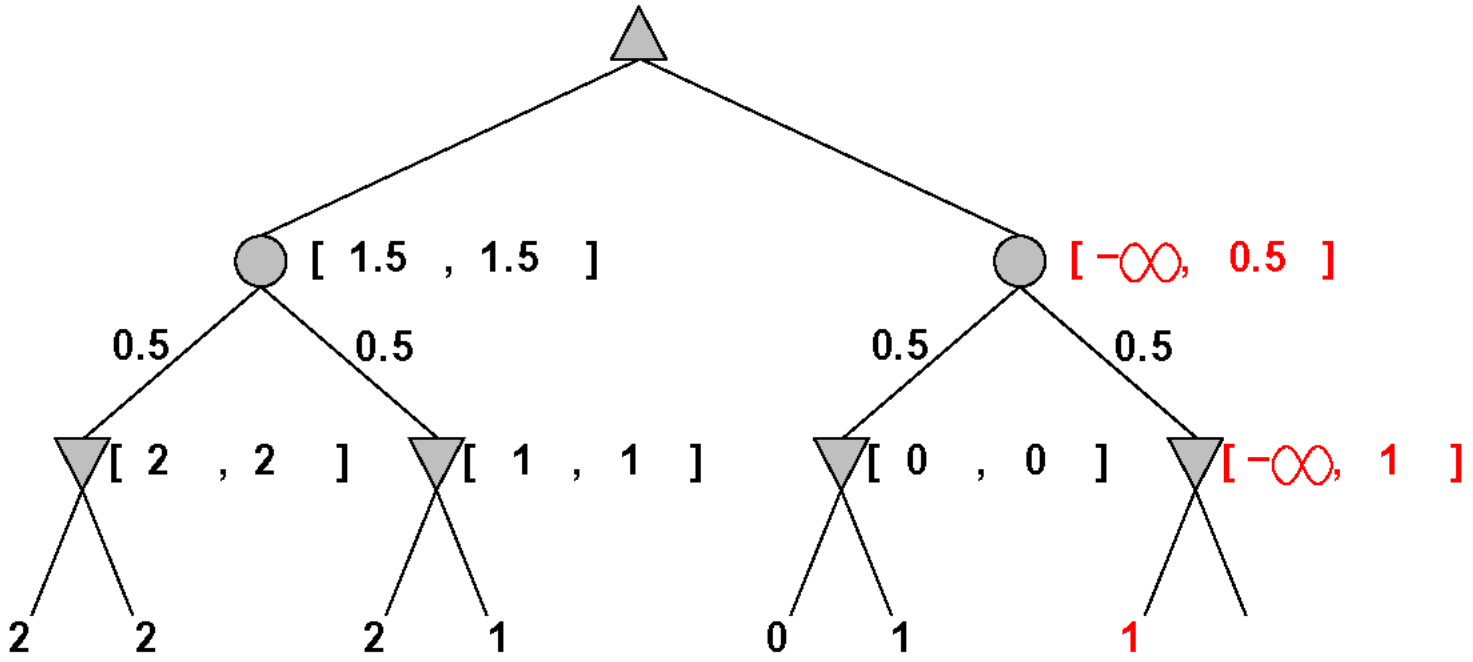
Pruning in nondeterministic game trees

A version of α-β pruning is possible:



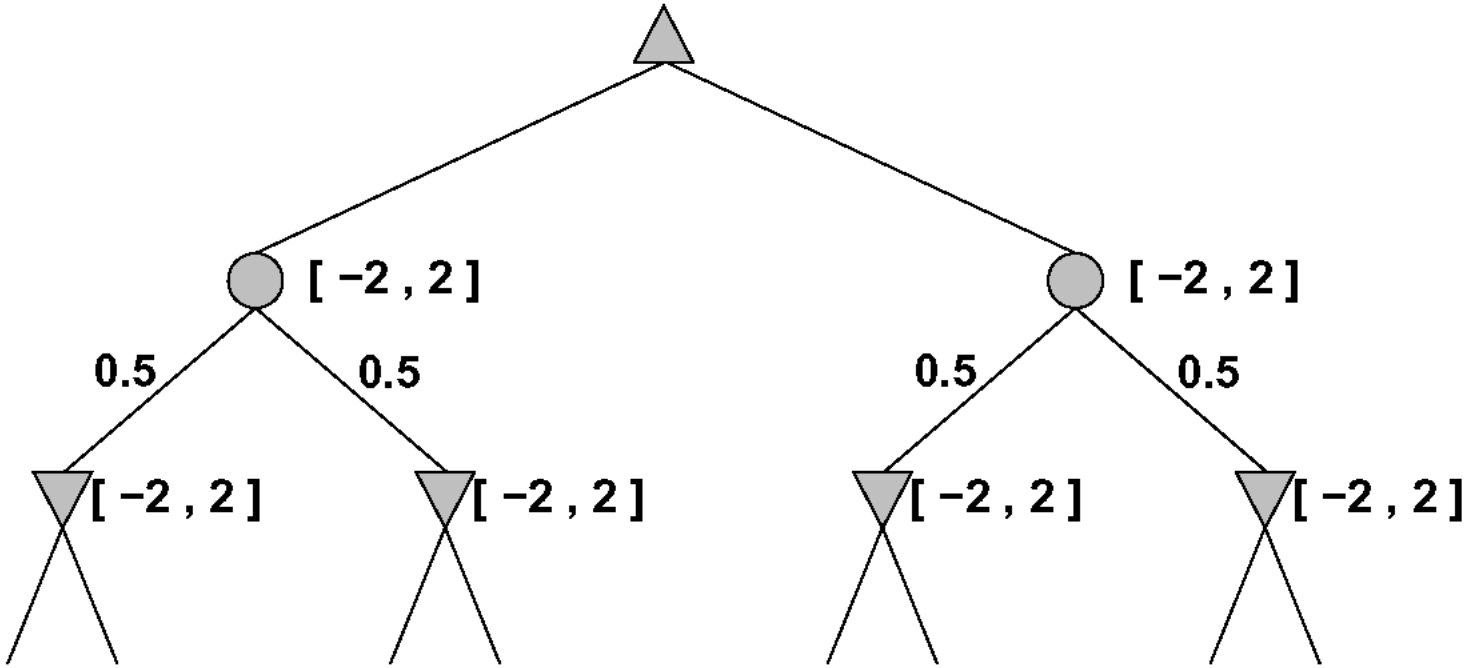
Pruning in nondeterministic game trees

A version of α-β pruning is possible:



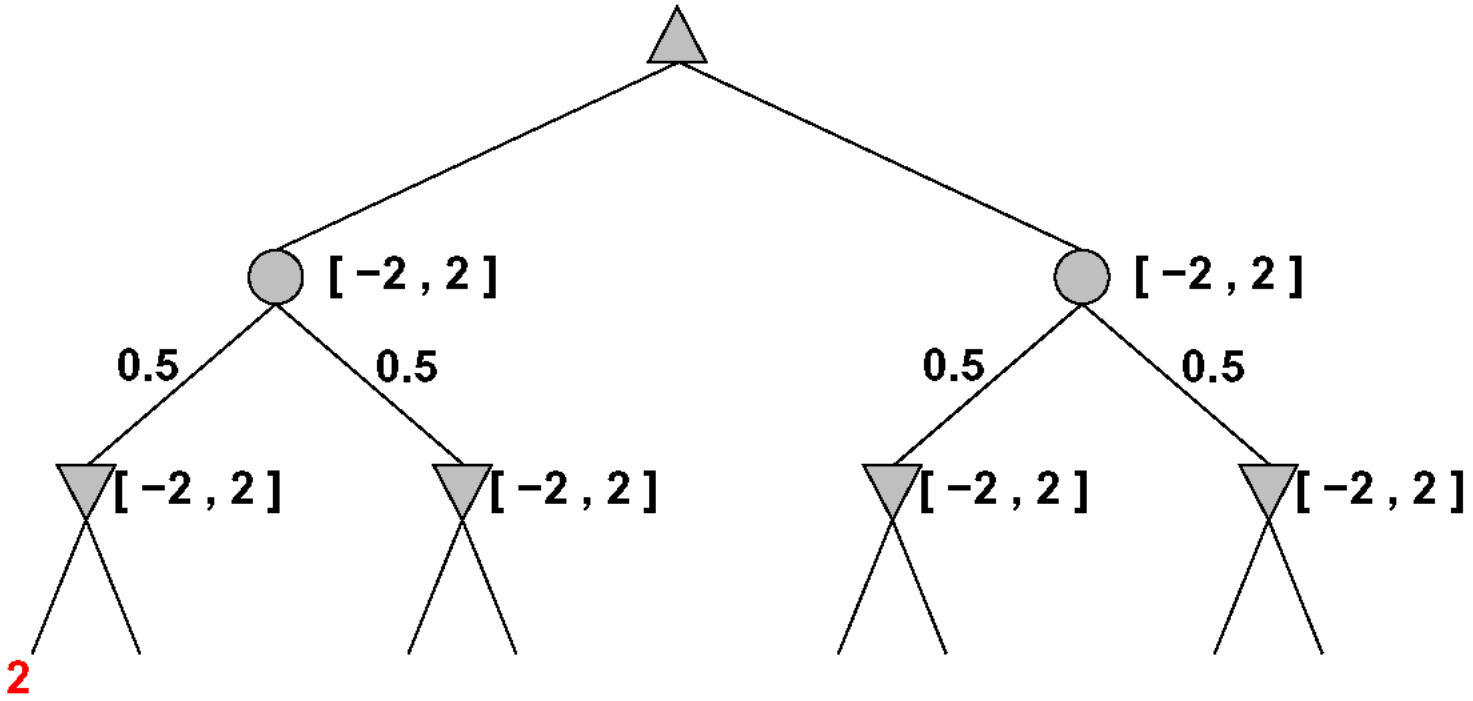
Pruning contd.

More pruning occurs if we can bound the leaf values



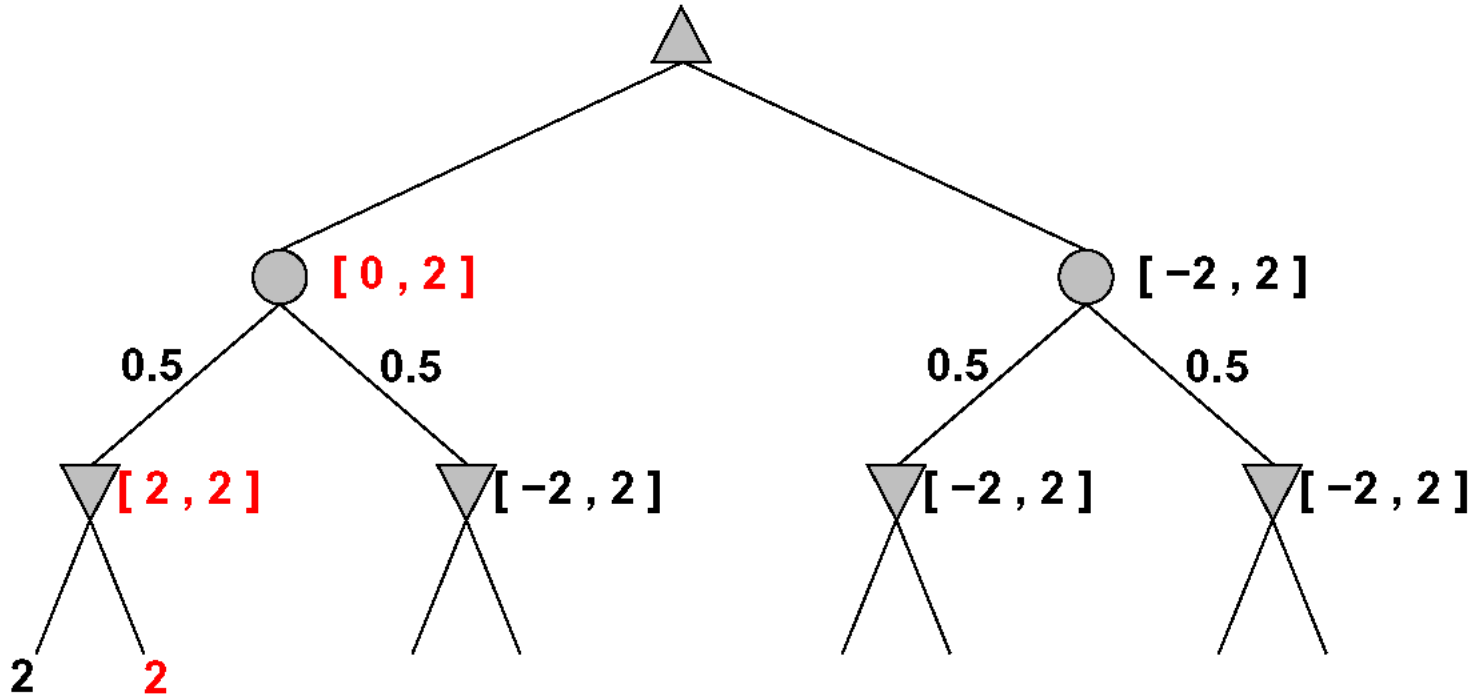
Pruning contd.

More pruning occurs if we can bound the leaf values



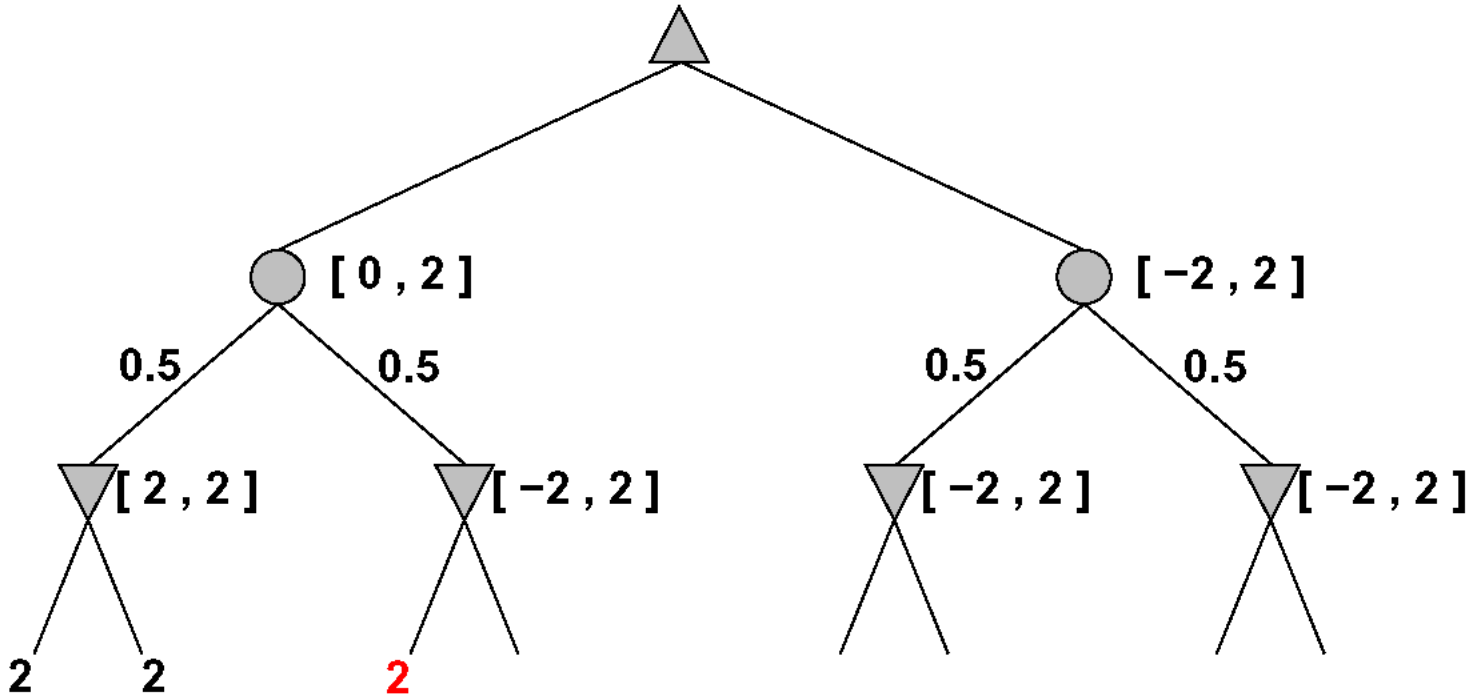
Pruning contd.

More pruning occurs if we can bound the leaf values



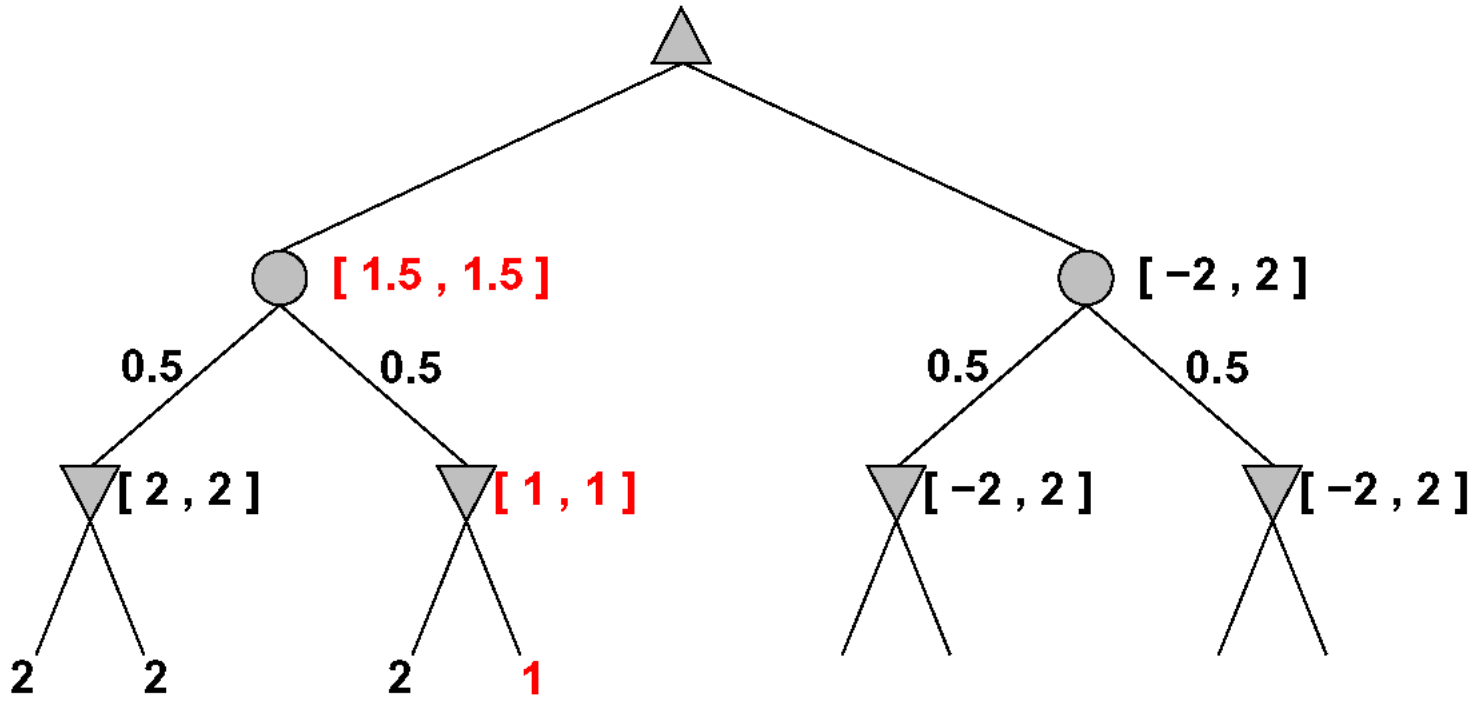
Pruning contd.

More pruning occurs if we can bound the leaf values



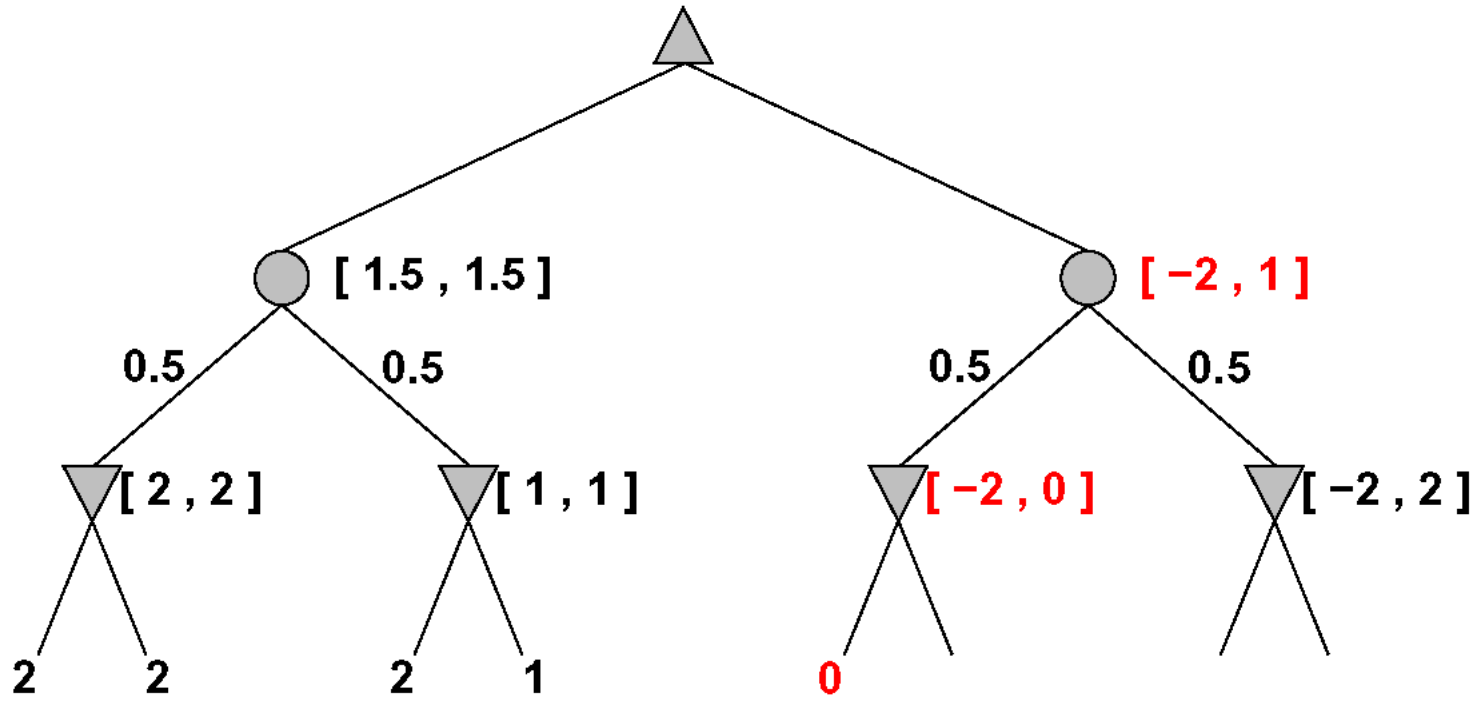
Pruning contd.

More pruning occurs if we can bound the leaf values



Pruning contd.

More pruning occurs if we can bound the leaf values



Nondeterministic games in practice

Dice rolls increase b: 21 possible rolls with 2 dice
Backgammon $\approx$ 20 legal moves (can be 6,000 with 1-1 roll)

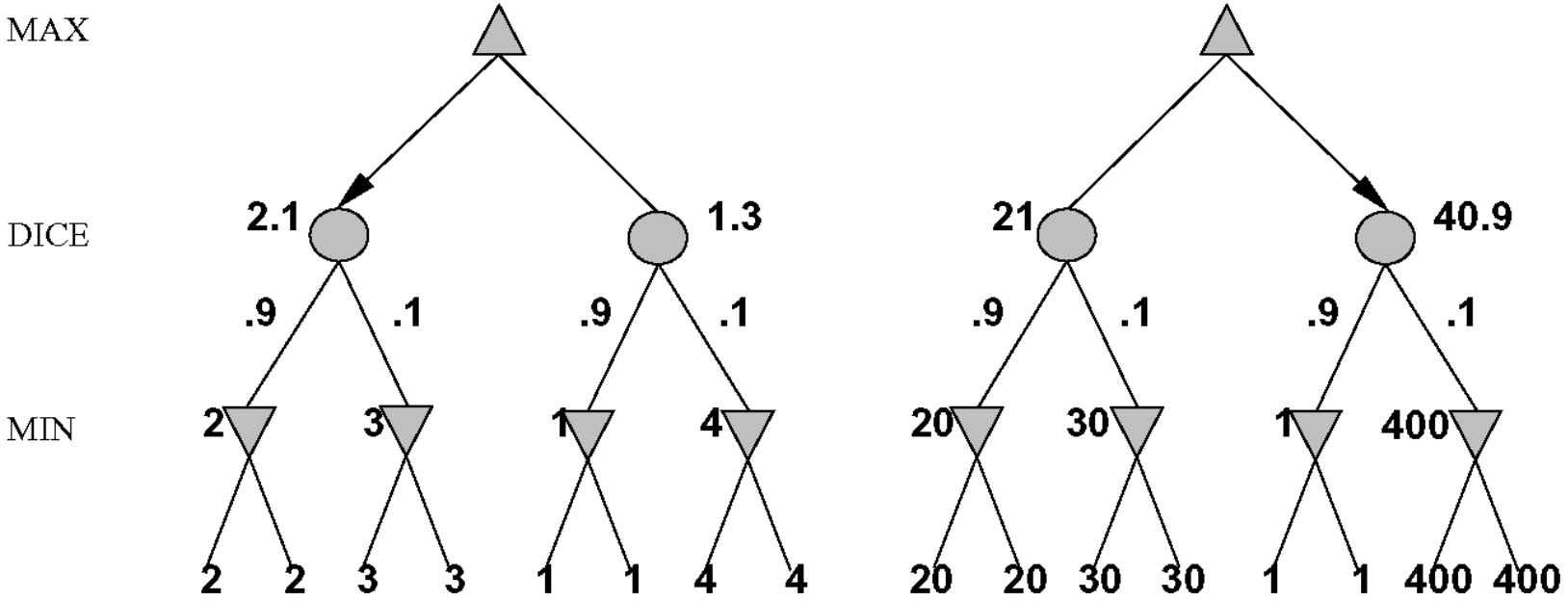
$$\text{depth } 4 = 20 \times (21 \times 20)^3 \approx 1.2 \times 10^9$$

As depth increases, probability of reaching a given node shrinks
$\Rightarrow$ value of lookahead is diminished

α–β pruning is much less effective

TDGammon uses depth-2 search + very good Eval
$\approx$ world-champion level

Digression: Exact values DO matter



Behaviour is preserved only by *positive linear* transformation of EVAL

Hence EVAL should be proportional to the expected payoff

Games of imperfect information

E.g., card games, where opponent's initial cards are unknown

Typically we can calculate a probability for each possible deal

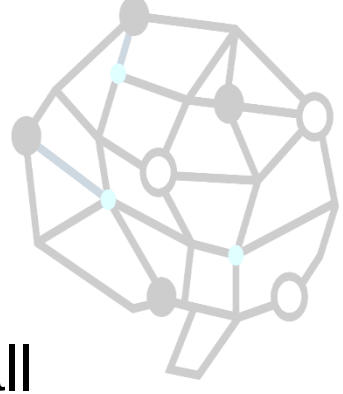
Seems just like having one big dice roll at the beginning of the game*

Idea: compute the minimax value of each action in each deal,
then choose the action with highest expected value over all deals*

Special case: if an action is optimal for all deals, it's optimal.*

GIB, current best bridge program, approximates this idea by
1) generating 100 deals consistent with bidding information
2) picking the action that wins most tricks on average

Solving games



- Game-theoretic value of a game
  - The outcome, i.e., win, loss or draw, when all participants play optimally.

Classication of games' solutions (L.V. Allis in 1994)

- Ultra-weakly solved
  - the game-theoretic value of the initial position has been determined.
- Weakly solved
  - for the initial position a strategy has been determined to achieve the game-theoretic value against any opponent.
    - The strategy must be efficient and practical in terms of resource usage.
- Strongly solved:
  - a strategy has been determined for all legal positions.

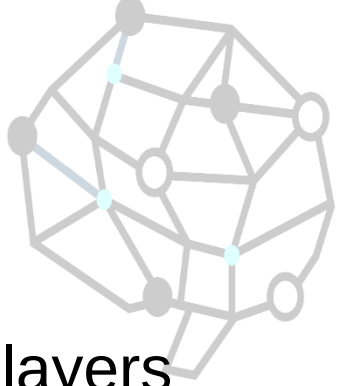
Complexity issue

- State-space complexity of a game
  - the number of all legal positions in a game.
- Game-tree (or decision) complexity of a game
  - the number of all leaf nodes in a solution search tree.
  - A solution search tree is a tree where the game-theoretic value of the root position can be decided.

Complexity for Tic-Tac-Toe

- Upper bound ⇒legal positions ⇒rotation and reflections
- State space
  - 3^9 (19683)⇒5478 ⇒765
- Game tree size
  - 9! ⇒255168 ⇒26830

Fair game



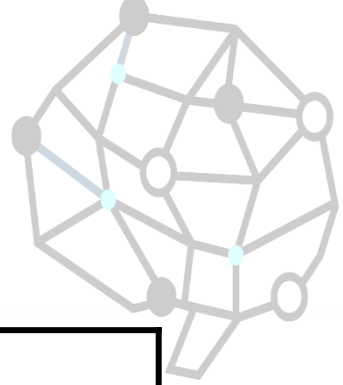
- A fair game
  - the game-theoretic value is draw and both players have roughly an equal probability on making a mistake

A double dichotomy of the game space

log log(state-space complexity) →

category 3 if solvable at all, then by knowledge-based methods	**category 4** unsolvable by any method
category 1 solvable by any method	**category 2** if solvable at all, then by brute-force methods

log log(game-tree complexity) →



Games	Log(state-space)	Log(game tree size)
Nine Men's Morris	10	12
Connect-four	14	21
Checkers	21	31
Othello	28	58
Chess	46	123
Chinese chess	48	150
Hex(11x11)	57	98
Shogi	71	226
Renju(15x15)	105	70
Go-Moku(15x15)	105	70
Go(19x19)	172	360

Games in practice

- Predictions for 2010 at 2000(Herik et.al.)

Solved	Over champion	World champion	Pro or Grand master	amateur
Go-Moku & Renju(15x15) Othello(8x8) Checkers(8x8)	Chess Backgammon	Go(9x9) Chinese chess Hex Amazons	Bridge Shogi	Go(19x19)

Games in practice

- Status

Solved	Over champion	World champion	Pro or Grand master	amateur
Go-Moku & Renju(15x15) Othello(8x8) Checkers(8x8)	Chess Backgammon Go(19x19) Go(9x9) Chinese chess Shogi Hex	Amazons Bridge		

Homework

- Implement a game bot for Tic-Tac-Toe by alpha-beta (min-max) algorithm.


THE END.